CS 1501 Notes:

Introduction:

Purpose:

Up until now, your classes have focused on how you COULD solve a problem. Here, we will start to look at how you SHOULD solve a problem.

Some Definitions:

1) **Offline Problem:**

We provide the computer with some input and after some time receive some acceptable output.

- It's **offline** b/c we're not providing input as we go…

2) **Algorithm:**

A step-by-step process for solving a problem or accomplishing some end.

3) **Program:**

An algorithm expressed in a language that the computer can understand.

An algorithm solves a problem if it produces an acceptable output on **EVERY** input.

Goals of the Course:

1) To learn to convert **non-trivial algorithms** into **programs**.
    a) Many "simple" algorithms can become **much more complicated**
    b) Algorithms can also be very complex to begin with, and their implementation must be considered carefully.
    c) Various issues will always pop up during **implementation**.
        ci) Such as **edge cases, Uncertainty about which Data Structures** to use, **debugging**, and **Control Flow, Encoding/Decoding, Libraries/Language** you use, etc.

2) To see and understand the differences in algorithms and how they affect the runtimes of the associated programs.
    a) Different algorithms can be used to solve the same problem!
    b) Different solutions can be compared using many **metrics**!
        bi) Runtime is a big one!!
            bij) Better runtime can make an algorithm **more desirable**.
            bijj) Better runtimes can sometime make a problem solution feasible (where it wasn't before)!

How do we Determine an Algorithm's Performance?

1) **Implement it and Measure its Performance!**

Are there any problems with this approach?

a) It could be computer-dependent!! (MAC vs. WINDOWS)

b) We have to implement it first, before we know how long it'll take!

c) Our inputs may be **Worst Case** for one algorithm, but the **Best Case** for another!

d) What if we made a mistake??? What if you find out that the algorithm takes too long???

2) **Algorithm Analysis**

a) Determine **resource usage** as a function of **input size**.

b) Measure **asymptotic performance**.

bi) Performance as input size increases to **infinity**.

Three Sum from the Textbook:

Problem: Given a set of arbitrary integers (could be < 0), find out how many distinct triples sum to zero.

Simple Solution: Triple For Loops!

Runtime: n^3

Big O:

It's an **upper bound on asymptotic performance**.

1) As input size grows to infinity, the function that represents **resource usage** (memory usage, etc.) **WILL NOT** exceed the specified function.

- Saying runtime is $O(n^3)$ means that as input size (**n**) grows to infinity, the actual runtime **WILL NOT** exceed n^3 (n^3 is the absolute max).

Wait…

Assuming the above definition…

1) Is ThreeSum $O(n^4)$?

2) What about $O(n^5)$?

3) What about $O(3^n)$?

If all of these are true, why was $O(n^3)$ what we jumped to?

- Because it's the tightest bound and it best captures the performance of the algorithm!

The Truth is… Big O isn't the Whole Story!

1) Big Omega

It's the **lower bound** on **asymptotic performance**!

$\Omega(n^3)$ means runtime will be no smaller than n^3.

- Could be $n, n^2, 3n^3$ …

2) Big Theta

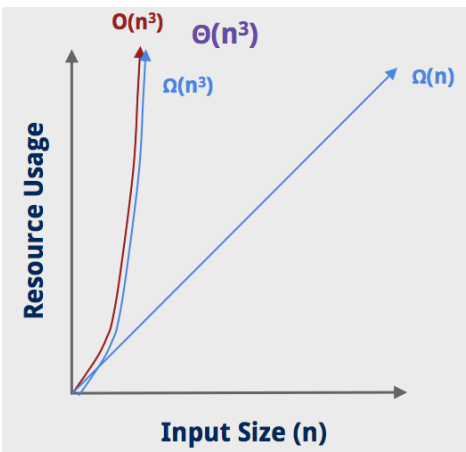
It's the **Upper** and **Lower Bound** on **Asymptotic Performance**!

- It's the exact bound!
- When Big Omega and Big O are equal, out pops Big Theta!

--------Big O--------

Big Theta

------Big Omega-----



In this graph, what we're saying is that the runtime is no faster than n, but no slower than n^3.

It's not **BEST CASE or WORST CASE**!

- It's simply an approximate/estimate bound!!

(LOOSE BOUNDS)

Formal Definitions:

1) $f(x)$ is $O(g(x))$ if constants "c" **(MULTIPLIER)** and x_0 **(THRESHOLD)** exist such that…

$$|f(x)| \leq c * |g(x)| \quad \forall x > x_0$$

- $g(x)$ will be an Upper Bound for $x > x_0$.
    - Any input size $> x_0$ will be an Upper Bound.

2) $f(x)$ is $\Omega(g(x))$ if constants "c" **(MULTIPLIER)** and x_0 **(THRESHOLD)** exist such that…

$$|f(x)| \geq c * |g(x)| \quad \forall x > x_0$$

- $g(x)$ will be a Lower Bound for $x > x_0$.

3) If $f(x)$ is $O(g(x))$ and $\Omega(g(x))$, then $f(x)$ is $\theta(g(x))$

c_1, c_2, and x_0 exist such that…

$$c_1 * |g(x)| \leq |f(x)| \leq c_2 * |g(x)| \quad \forall x > x_0$$

4) We might also see $f(x) \in O\big(g(x)\big)$ or $f(x)$ is $O(g(x))$

- $O(g(x))$ is the Set of all Functions that satisfy the Big O definition.

Mathematically Modelling Runtime:
Runtime is primarily determined by two factors…
1) The Cost of Executing each Statement
- Which is determined by the Machine Used, and the Environment that's Running on the Machine.

2) The Frequency of the Execution of each Statement
- Which is determined by the Program and the Input.

Three Sum: Order of Growth:
1) **Upper Bound:** $O(n^3)$
2) **Lower Bound:** $\Omega(n^3)$
3) Hence, $\theta(n^3)$

Tilde Approximations:
For ThreeSum, $\sim n^3/6$
- This is pretty pointless, unless the Algorithms are similar!

Common Orders of Growth:
1) **Constant** – 1
2) **Logarithmic – log n**
3) **Linear – n**
4) **Linearithmic – n log n**
5) **Quadratic** - n^2
6) **Cubic** - n^3
7) **Exponential** - 2^n
8) **Factorial – n!**

Ascending…
Best -> Worst

How can we ignore Lower Order Terms and Multiplicative Constants?
Remember, this is Asymptotic Analysis!
- As n grows in size to infinity, the differences become really small!

Quick Algorithm Analysis:
Ignore multiplicative constants and lower terms!

Easy to get Theta for 3 Sum?

Why do we bother with **Big O** and **Big Omega**?

1) It's not always like this!!

- Upper Bounds are usually easier, obvious, and tight.
- Lower Bounds are tighter.

More on Three Sum:

Is there a better way?

1) What if we sorted the array first?

- Pick two numbers, and then binary search for the minus sum.

Ex: a[i] + a[j] -> Binary Search for −(a[i] + a[j])

2) Now the runtime is $O(n^2 log(n))$

3) But… what if the data isn't sorted???

Sorting:

Given a list of n items, place the items in some **LOGICAL ORDER**.

- Could be ascending, descending
- Numerical, alphabetical, etc.

Prerequisites (Remember Generics?):

With sorting algorithms, we're gonna involve the following methods:

1) **less**

- This will compare items, and return true, if **v is less than w**.

```
<T extends Comparable<? super T>>
boolean less(T v, T w)
{
        return (v.compareTo(w) < 0);
}
```

v.compareTo(w) returns…

1) -1 when v < w
2) 0 when v = w
3) 1 when v > w

2) **exchange**

- This will exchange the values at index i and j.

```
<T> void exchange(T[ ] a, int i, int j){
        T swap = a[i]
        a[i] = a[j]
        a[j] = swap;
}
```

Bubble Sort:
This sorting algorithm sucks.

Here's how it works..
1) We simply go through the entire array, comparing pairs of items, and swap them if they're out of order. (We're looking for inversions!)
- And we repeat until we make it through the array with 0 swaps.

Runtime:
$\theta(n)$: Best Case, we go through the entire array and find no inversions.
$\theta(n^2)$: Worst Case, we go through the entire array, and have to swap ALL ITEMS.

Pseudocode:

```
void bubbleSort(Comparable [ ] a)
{
        boolean swapped;
        do
        {
                swapped = false;
                for(int j = 1; j < a.length; j++)
                {
                        if(less(a[j], a[j-1])
                        {
                                exch(a, j-1, j);
                                swapped = true;
                        }
                }
        }while(swapped);
}
```

Bubble Sort Example:

5	3	4	1	10

1) 3 and 5 is an inversion, so we swap 5 and 3…

3	5	4	1	10

2) 5 and 4 is an inversion, so we swap 5 and 4…

3	4	5	1	10

3) 5 and 1 is an inversion, so we swap 5 and 1…

3	4	1	5	10

4) 4 and 1 is an inversion, so we swap 4 and 1…

3	1	4	5	10

5) 3 and 1 is an inversion, so we swap 3 and 1…

1	3	4	5	10

6) Now the list is sorted!!

1	3	4	5	10

"Improved" Bubble Sort:

Add a new variable: **to_sort = a.length;**

1) We can keep track of the elements to look at…

2) We look at one fewer element on each successive swap!!

```
void bubbleSort(Comparable [ ] a)
{
        boolean swapped;
        int to_sort = a.length;
        do
        {
                swapped = false;
                for(int j = 1; j < a.length; j++)
                {
                        if(less(a[j], a[j-1])
                        {
                                exch(a, j-1, j);
                                swapped = true;
                        }
                }
                to_sort--;
        }while(swapped);
}
```

But… this is still $\theta(n^2)$!

Comparison-Based Algorithms:

"*The best runtime a Comparison-Based Algorithm can do is $\theta(n\ log(n))$, being either Mergesort, or Quicksort*"

- Mergesort uses n space to allocate an **auxiliary array**.

Sorting without Comparison:

1) Look at the **Least-Significant Digit (LSD)**
2) Group the numbers with the same digit.
- Maintain relative orders (**BUCKETS**)
3) Place groups back into the array together.
- I.e., all the 0's, all the 1's, all the 2's, etc.
4) Repeat for increasingly significant digits.

LSD Radix Sort Example:

012	103	212	023	003	113	121	200

1) Look at the One's digit… place the digits in buckets based on ascending order…

0's in 1's Place

1's in 1's Place

2's in 1's Place

3's in 1's Place

2) Now put them back in the array, while maintaining the same order we sorted by…

200	121	012	212	103	023	003	113

3) Now look at the Ten's digit… place the digits in buckets based on ascending order

0's in 10's Place

1's in 10's Place

2's in 10's Place

4) Now put items back in the array, while maintaining the same order we sorted by:

200	103	003	012	212	113	121	023

5) Now look at the Hundred's digit… place the digits in buckets based on ascending order…

0's in 100's Place

1's in 100's Place

2's in 100's Place

6) Now put the items back into the array, while maintaining the same order we just sorted by…

003	012	023	103	113	121	200	212

Now the array is sorted!!!

Radix Sort Analysis:
Runtime: $\theta(wn)$
If "w" is fixed, we get $\theta(w)$
$\#\ of\ items\ (n) * length\ of\ largest\ item\ (w)$
- For example, 001's length is w = 3.
- n is the # of inspections!

Memory: $w + n$

But… are Non-Comparison Based Sorting Algorithms better than the Best-Case Comparison Based Sorting Algorithm, $n\ log(n)$**?**
They both have their drawbacks…
1) If we're dealing with things with short lengths: **Radix Sort** is better!
2) If we're dealing with things with long lengths: **Merge Sort** is better!

Note:
When "w" is close to **log(n)**, Mergesort and Radix Sort have approximately the same Runtime!

This is called **Radix Sort** b/c we used **Bases**.
- Radix is just another word for **Base**, so since we used Base 10, we could've just said **Radix 10**.

Is this an In-Place Algorithm?
I.e., is it good with only the memory allocated in the initial array?
Nope! We have to move the items to an auxiliary array, then copy back!

Is Radix Sort a Stable Algorithm?
I.e., do we maintain some order?
Yes, we maintain buckets in the same order!!

What is the Most Efficient Way to Sort 1 Million 32-bit Integers?
Well… 1,000,000 32 bit integers don't take up a lot of space…
- 4 MB?

But… what if we needed to sort 1 TB of numbers?
1) It won't all fit in memory!!
2) We've been assuming that we were performing **internal** sorts!
- Everything is in memory!
3) We now need to consider **external** sorting!
- Where we need to write to disk!

Hybrid Merge Sort:
1) Read in the amount of data that will fit in memory.
2) Sort in place!
- I.e., via Quick Sort.
3) Write the sorted chunk of data to disk!
4) Repeat until all data is stored into sorted chunks!
5) Merge all the chunks together!

Searching:

Searching Review: Searching through a Collection

Given a collection of keys, "C", how do we search for a given key, k?

Data Structure	Runtime (Search)
Arrays (Unsorted)	$\theta(n)$: **Exhaustive**
Arrays (Sorted)	$\theta(log(n))$: **Binary Search**
Linked List (Unsorted)	$\theta(n)$: **Exhaustive**
Linked List (Sorted)	$\theta(n)$: **Can't do Binary Search (Random Array Access is difficult in a LL)**
Binary Search Tree (Binary Search Stick)	$\theta(n)$: **Have to traverse through entire "stick"**
Binary Search Tree (Balanced)	$\theta(log(n))$

Why do we Prefer one Algorithm over the Other?

1) Memory!
2) Ease of Implementation!
3) If we don't care about dynamism…

Symbol Tables:

Symbol Table: An ADT that links a **unique key** to a **value**.

1) We can search for the value using the key.
2) AKA Associative Arrays

DEF: A Symbol Table is an **INTERFACE** for Key-Value pairs that supports 2 operations…

1) **put():** Inserts a Key Value pair into the table
2) **contains():** Is value paired with a key?

KEYS: LIKE ARRAY INDICES

VALUES: LIKE ARRAY ENTRIES

A Closer Look:

BinarySearchST.java BST.java

Present Symbol Tables

Sorted Array Unsorted Array

But… can we do better?

They both depend on comparisons against other keys!!

- I.e., **k** is compared against other keys in the Data Structure!

There are 4 Options at Each Node in a BST:

1) Is the reference node NULL? Then "k" is not found!!
2) "k" == Current Node's key? "k" found!
3) "k" < Current Key? Continue to Left Child…
4) "k" > Current Key? Continue to Right Child…

Can we make this faster by doing something similar to Radix Sort?

Digital Search Trees (DSTs):

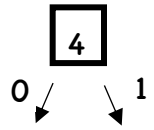
Instead of doing **LESS THAN/GREATER THANS**, let's go **LEFT** or **RIGHT** based on the bits of the key, so again, we have the following 4 options…

1) Is the reference Node NULL? Then "k" is not found!
2) "k" = Current Node's key? "k" is found!
3) Is the Current Bit of "k" = 0? Continue to the Left Child!
4) Is the Current Bit of "k" = 1? Continue to the Right Child!

DST Example:

We want to insert 4….

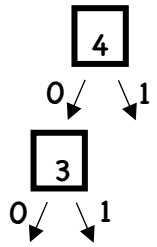
4 is 0100 in Binary, and since the Root Node is currently NULL, we make 4 the root now…



Now we want to insert 3…

3 is 0011 in Binary, and since the Root Node is 4, we look at the MSD of 3.

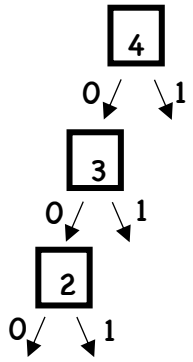
- Since the MSD of 3 is 0, we go to the left!



Now we want to insert 2…

2 is 0010 in Binary, and since the Root Node is 4, and the MSD of 2 is 0, we go left, since 3 is already there, we look at the 2nd MSD of 2, since it is 0, we append 2 to the left of 3!

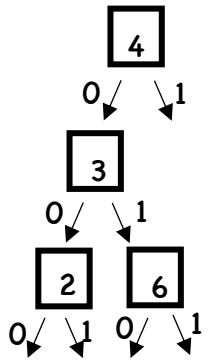
- Since the 2nd MSD of 2 is 0, we go to the left!



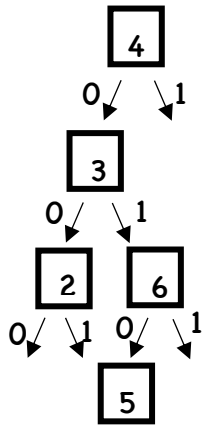
Now we want to insert 6…

6 is 0110 in Binary, and since the Root Node is 4, and the MSD of 6 is 0, we go left, since 3 is already there, we look at the 2nd MSD of 2, since it is 1, we append 6 to the right of 3!

- Since the 2nd MSD of 6 is 1, we go to the right!



Now we want to insert 5…
5 is 0101 in Binary, and since the Root Node is 4, and the MSD of 5 is 0, we go left, since 3 is already there, we look at the 2nd MSD of 2, since it is 1, we go left, since 6 is already there, we look at the 3rd MSD of 2, since it is 0, we go to the left of 6! - Since the 3rd MSD of 5 is 0, we go to the left!



Searching:
Say we want to search for 3…
Since 3 is 0011, we follow the paths of the bits, and if nothing is there… :(

Path:
Start at root node…
MSD of 3 is 0, go left, look at value, we found it!

Say we want to search for 7…
Since 7 is 0111, we follow the paths of the bits…

Path:
Start at root node…
0 -> go Left, 1 -> go Right, 1 -> go Right, but there's nothing there!!!

Analysis of DST:
Runtime:
If the Tree is balanced: $\theta(log(n)$

But, usually, it's: $\theta(bn)$
b: Bit Length
1) Imagine I go through as many bits and **NOT FIND IT :(**

Example:
If bit length is 4…
4 -> 0100 instead of 100

But it's $\theta(b)$ (But remember: $\theta(b) \neq \theta(1)$) if we can bind the length of the keys (E.g. no more than 12 bits!!)

Memory: $b + n$

But, uh… we still end up doing **A LOT** of compares based on Bit Length (Like with Radix Sort)

We're still comparing against **FULL KEYS**!
- "What if we have 1000 characters vs 1000 characters and they only differ at the last character?"

Radix Search Tries (RSTs):
Trie as in Re**TRIE**ve (pronounced **TRY**)

A trie only uses the **REPRESENTATION OF THE KEY!!**

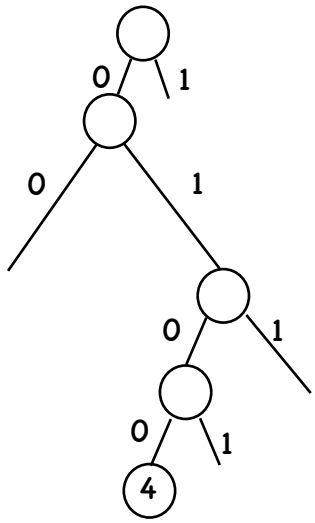
Instead of storing keys as Nodes in the tree, we store them implicitly as **PATHS** down the tree.
1) Interior nodes of the tree only serve to direct us according to the **Bit String** of the **Key**.
- I.e., we make them **NULL** until we reach the end.

2) Values can then be stored at the end of the Key's Bit String path!
- Stored at the bottom of the path (Instead of along the way)

RST Example:

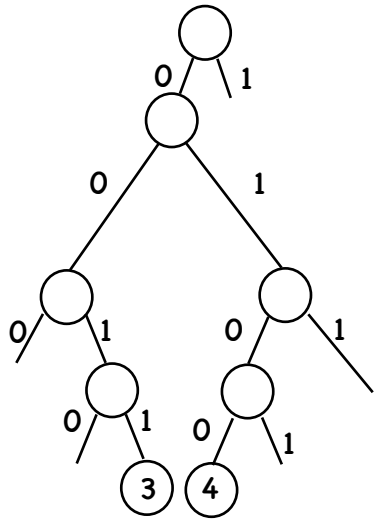
We want to insert 4…

4 is 0100 in Binary, so we create a path based on the Bit String, if the bit we're looking at is a 1, go right, otherwise we go left…

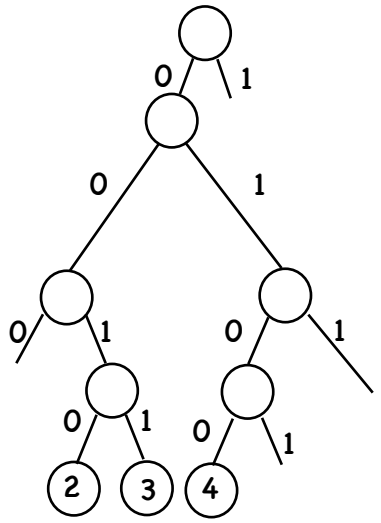


We want to insert 3…

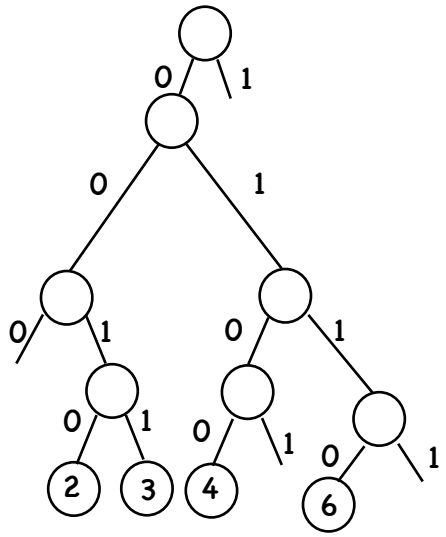
3 is 0011 in Binary, so we create a path based on the Bit String, if the bit we're looking at is a 1, go right, otherwise we go left…



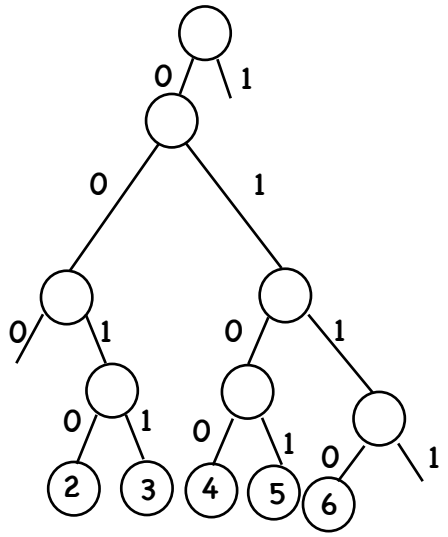
We want to insert 2…
2 is 0010 in Binary, so we create a path based on the Bit String, if the bit we're looking at is a 1, go right, otherwise we go left…



We want to insert 6…
6 is 0110 in Binary, so we create a path based on the Bit String, if the bit we're looking at is a 1, go right, otherwise we go left…



We want to insert 5…
5 is 0101 in Binary, so we create a path based on the Bit String, if the bit we're looking at is a 1, go right, otherwise we go left…



Searching:
We want to search for 3…
Well, 3 is 0011 in Binary, so we just follow the path!!
- There's something at the end of this path, so we found it!

We want to search for 7…
Well, 7 is 0111 in Binary, so we just follow the path!!
- This path doesn't exist, so we didn't find it!

Runtime:
$\theta(b)$ in all cases!

Memory: Much more than DSTs… (More Nodes!!) :(

But… would RSTs work well for other Data Types?
1) Characters? Use the **ASCII TABLE!**
2) Strings? Well… they're just sequences of characters…

Large Branching Factor Tries:

1) In our binary-based RST, we considered one bit at a time…

2) What if we applied the same method to characters in a string?

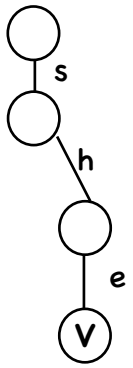
- What would this new structure look like?

3) Let's try inserting the following strings into a trie:

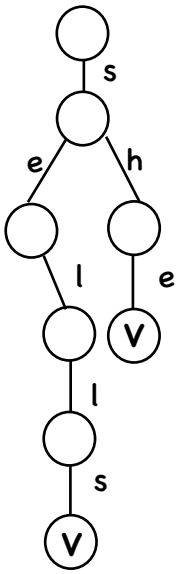
she, sells, sea, shells, by, the, sea, shore

RST String Example:

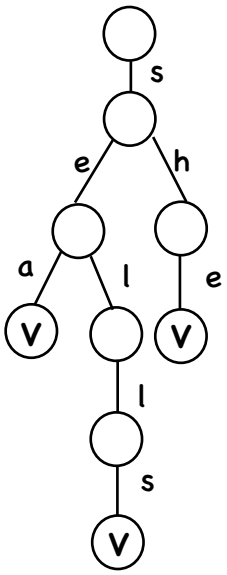
We want to insert **she…**



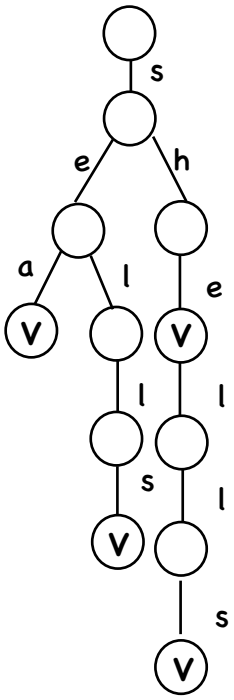
Now, we want to insert **sells…**



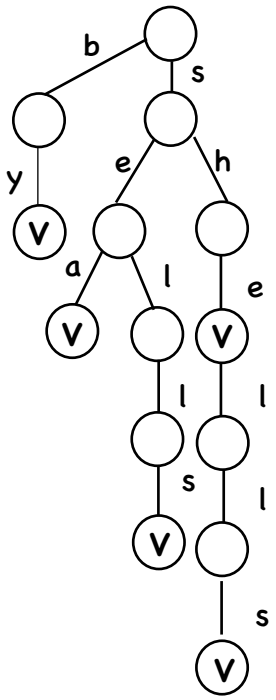
Now, we want to insert **sea**...



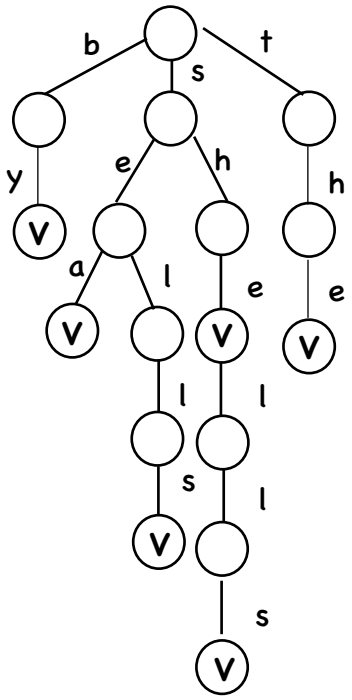
Now, we want to insert **shells**...



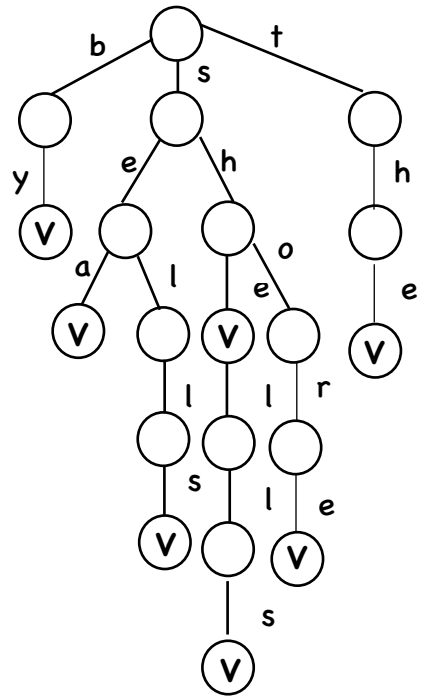
Now, we want to insert **by**...



Now, we want to insert **the**...



Now, we want to insert **shore...**



Runtime: $\theta(w)$
- Assuming Children are stored in an array.
(Quick Random Access)

Memory: $wR + n$

Further Analysis:
1) **Miss Times**
- Require an average of $log_R(n)$ nodes to be examined.
Where R is the size of the alphabet being considered.

What is the average # of checks with 2^20 keys assuming 8 bits at a time in an RST:

$$log_{256}(2^{20}) = 2.5$$

- With 2^20 keys, we require an average of 2.5 nodes to be examined at a time.

Implementation Concerns:

TrieST.java

- Implements an R-way Trie

Basic Node Object:

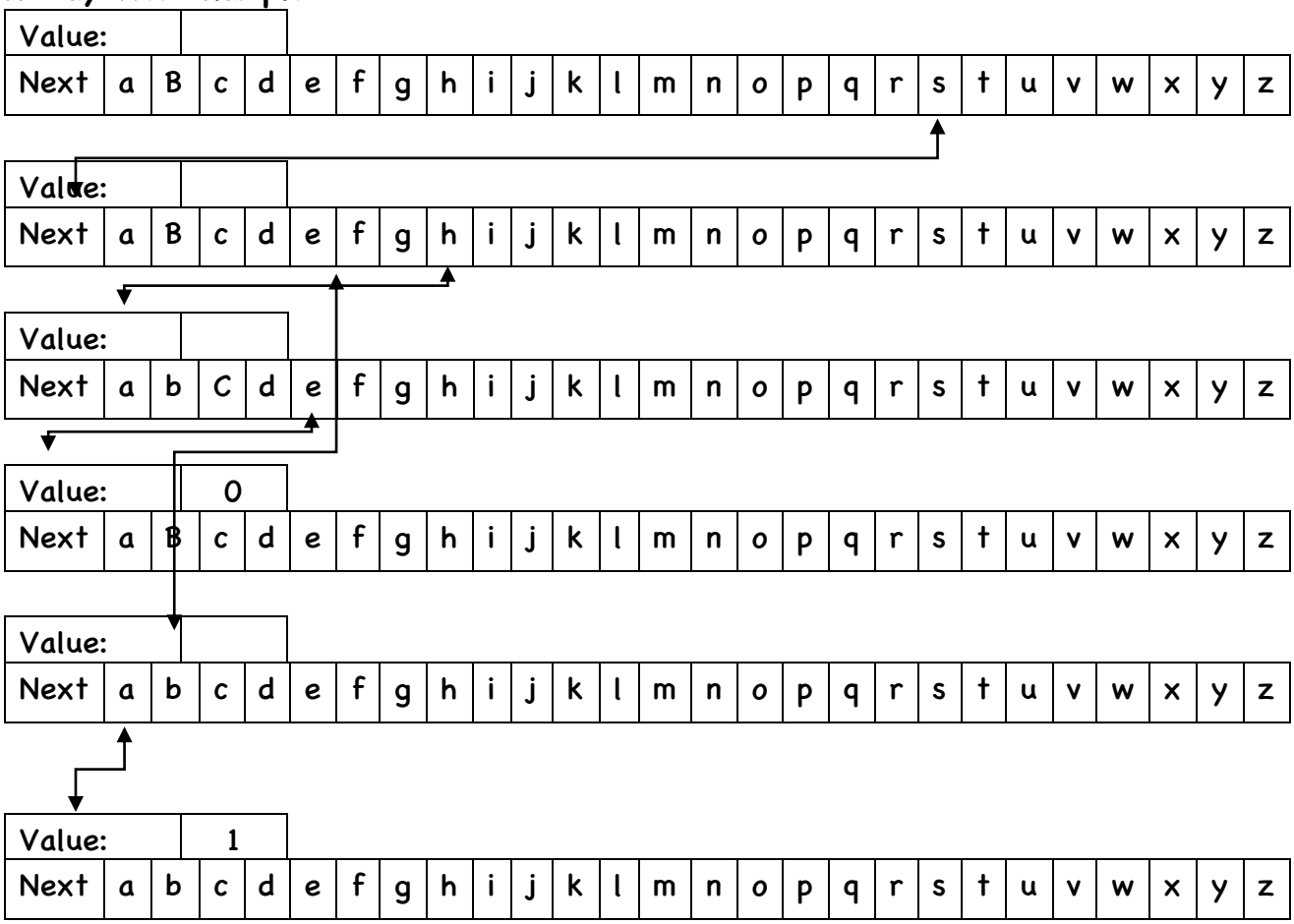
```
private static class Node
{
        private Object val;
        private Node[ ] next = new Node[R];
}
```

Where **R** is the branching factor.

Non-null value means we have traversed to a valid key!

Note: Keys are not directly stored in the Trie at all!

R-way Trie Example:



So, what's the catch?
Memory!
1) Considering 8-bit ASCII, each node contains 2^8 references!!
2) This is especially problematic as in many cases, a lot of this space is wasted!!
- Common paths or prefixes, e.g., if all keys begin with **"key"**, that's 255 * 3 wasted references!
- At the lower levels of the trie, most keys have probably been separated out, and reference lists will be sparse
(I.e., they have already been identified)

De La Briandais Tries (DLBs):
Replace the **.next array** of the R-way trie with a **Linked-List**!!

We're going to have some sort of **TERMINATOR CHARACTER** to indicate the end of a string, like a **^**.

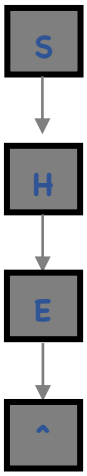
DLB: Every individual node has just **ONE CHILD**.

The **NEXT CHILD**, is represented as **SIBLINGS**. ("H" points to "E")
- Represented as a linked chain to other children!

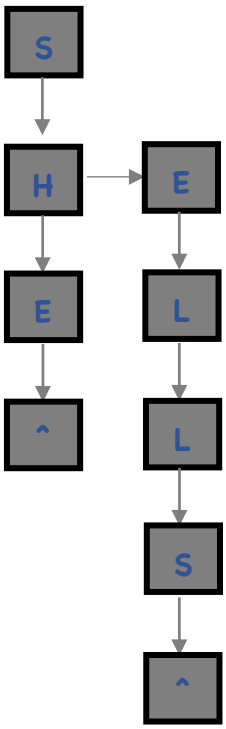
There are two references:
1) **CHILD** (Represented Vertically)
2) **SIBLINGS** (Represented Horizontally)

DLB Example:

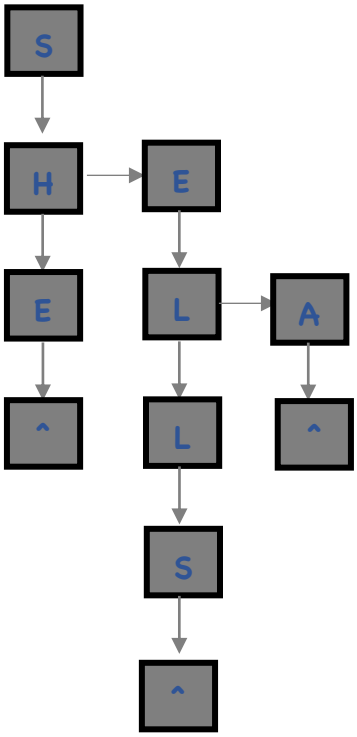
We want to add **she…**



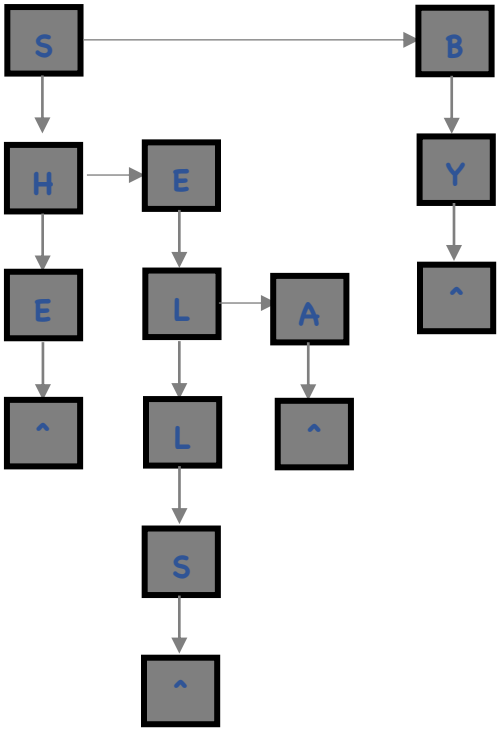
Now, we want to add **sells…**we follow the **s**, and we would usually append a child node there, but since "**h**" is already there, we append "**e**" as a sibling node to "**h**".



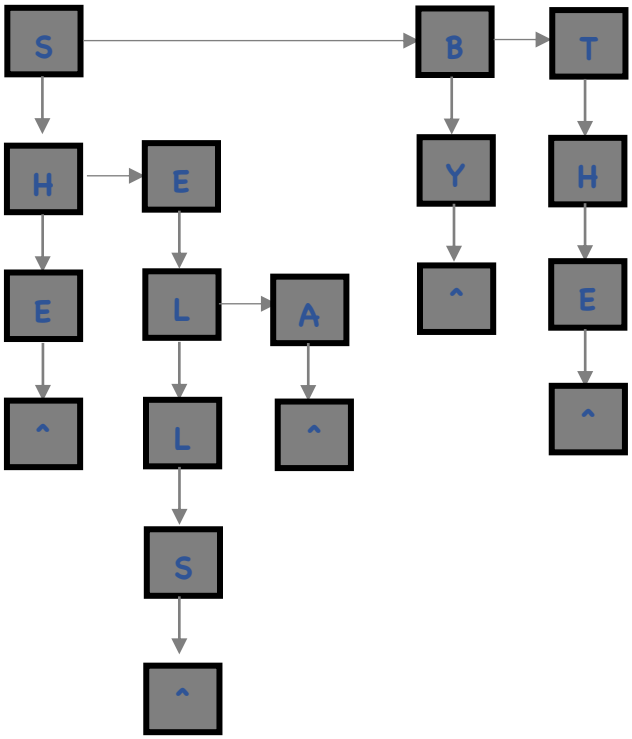
Now, we want to add **sea…** we follow the **s**, and then the **e**, and we would usually append **a** as **CHILD NODE** to **e**, but since **l** is already there, we append **a** as a **SIBLING NODE** to **l**.



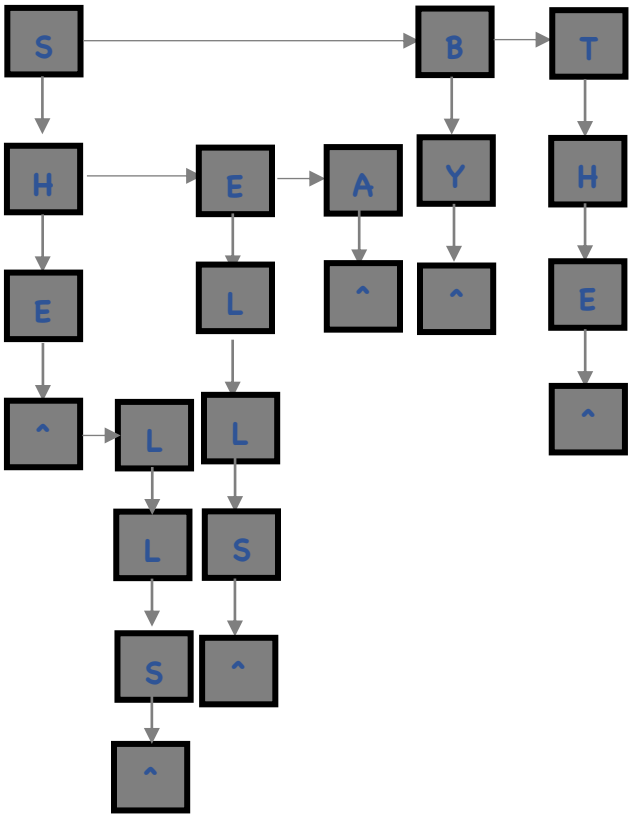
Now, we want to add **by…** since we have no nodes starting with **b**, we would usually append the **b** where the **s** is, but since **s** is already there, we append **b** as a **SIBLING NODE** to **s**.



Now, we want to add **the…** since we have no nodes starting with **t**, we would usually append the **t** where the **s** is, but since **s** is already there, and there's already a **SIBLING NODE** to s, we append **t** as a **SIBLING NODE** to **b**.



Now, we want to add **shells…** since we already have **she** in the DLB, we need to add **lls** as a **SIBLING NODE** to the **TERMINATOR CHARACTER**.



DLB Analysis:
How does DLB performance differ from R-way Tries?
1) We've lost random access :(
2) We're using less memory!!

Runtime:
$\theta(Rw)$: Worst Case, we're looking at every level until we get the String we want.
- **Alphabet * Characters in Keys**

So…which should you use?
Well… it depends.

If the data is dense, we'll use an **R-way trie** (Random/Short)

If the data is **sparse**, we'll use a DLB (Long/Not Random)

Note: We should never get $\theta(wR)$, if you did, you're using the **WRONG DATA STRUCTURE**.
- We should expect w/sparse data, $\theta(w)$.

Searching:
So far, we've only assumed that each search would only look for the presence of a whole key!!

But…what about if we wanted to know if our search term was a ***prefix*** to a valid key???

Welp…
BSTs -> Not that great for this
DST -> Uh… not much better

RECURSION:
1) NOT FOUND (NODE IS NULL)
2) YES, we found a prefix, can I keep going?
3) Yes, we found a prefix, and it's a full key!
4) Something used to be there, but it's no longer there :(

B-Trees:
It's a generalized BST!
1) Not restricted to 2 children.
- Up to **k** children

The Problem:
1) So far, we've only talked about several approaches to search through a set of keys for a value.
- Several implementations of Symbol Tables!
2) We always assumed that all of the STs were storing keys and values in memory!
3) But, umm, what if Data needs to be stored on Disk?
- What should we do differently?

Consider the following example:
1) You are writing software that will be used to store records of online store transactions, each with a unique ID…

2) You want to store the Transaction Records on **DISK**.
- You expect a large volume of transaction records.
- You want the Transaction Records stored in a non-volatile memory?

3) How can you still efficiently search for a given transaction by its ID?

Disk Storage:
1) The data stored on disk is grouped into **blocks**.
- Typically, **4KB**.

2) I/O to the disk is performed at the block level.

3) To read a file from the disk, the OS will fetch all of the blocks that store some portion of that file and read the data from each block.

B-Trees:

1) A **B-Tree** operates similarly to a BST, but it's not limited to a branching factor of 2.

2) The order of a B-Tree determines the max branching factor.

- Invariants for an order **M** B-tree:

a) Nodes have a max of **M** children.

b) Interior nodes have a min of $\lceil M/2 \rceil$ children.

(Nodes that are not the root or leaves)

Corollary: All interior nodes must be at least half full.

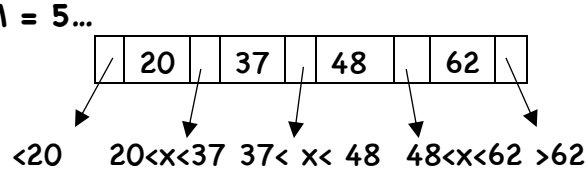
c) The root has **at least two children** if it is not a leaf node.

d) Non-leaf nodes with **k** children have **k – 1 keys stored**.

e) All leaves appear on the same level.

Here's how the **ROOT of a B-Tree could look like:**

Let M = 5…



Inserting into a B-Tree:

1) Start with a Single Node.

2) Add Keys until the Node fills.

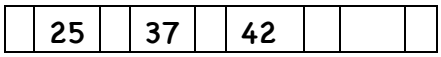
- I.e., contains **M-1** keys, and has **M** children.

3) In adding the **Mth key**, split the node in two.

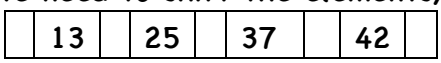
- Pull one key up to the parent node (**PROMOTION**).
  - This could cause us to create a new parent node, cascade and split the parent, etc.

B-Tree Insertion Example:

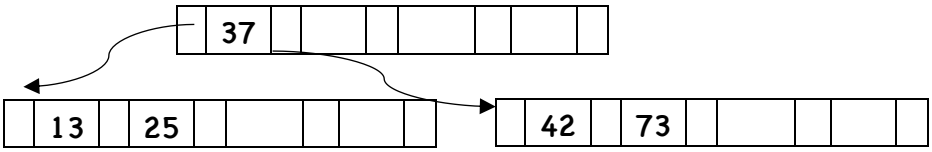
M = 5



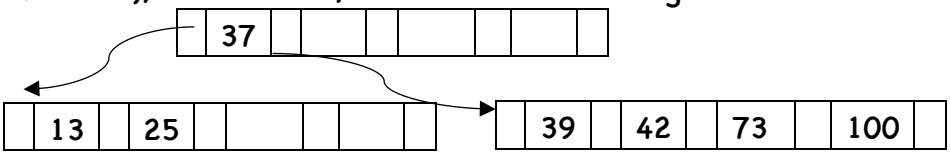
Adding 13, we need to shift the elements, so that 13 ends up the right position:



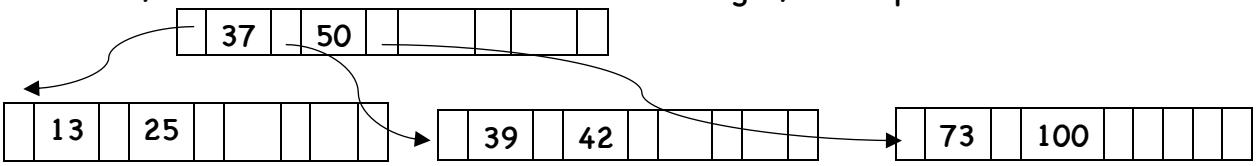
Add 73, take the median of the numbers, and promote the median as the new root node…



Insert 100 and 39, we look at the right, and insert 39 to the left of 42 (shift elements), and for 100, we insert it to the right of 73…

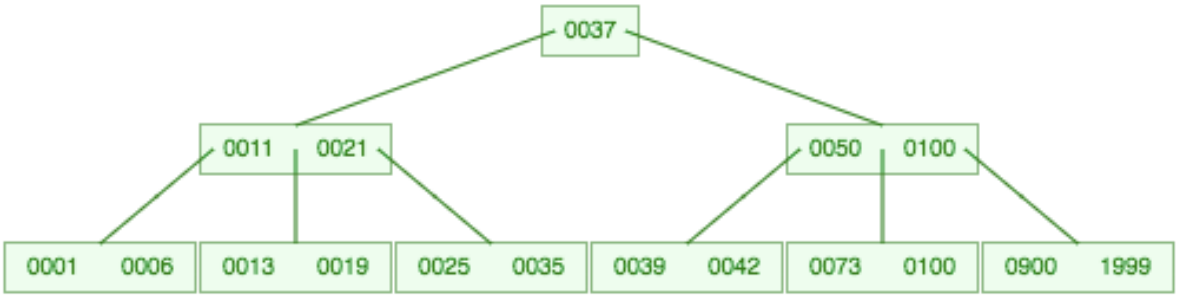


Insert 50, it turns out to be the median of the right, so we promote it as the root!



What if the root node splits?

Insert 21, 3, 16



Anything greater than 21 and less than 37

Anything greater than 37 but not less than 50

OK, that's cool and all, but how does this help us store transaction records?
1) It helps us see how to store IDs as keys, but what about what about full records of a sale transaction?
- ID, customer information, price, item purchased, how many purchased, etc…

B-Tree Analysis:
What's the Runtime?
1) **Search:** $\theta(\log_M(n))$ = $\theta(log(n))$ like a balanced BST!
2) **Insert:** $\theta(log(n))$
- We only need to look at 2 nodes at a time!
3) **Deletion:** $\theta(log\ (n))$
- Only look at two nodes when considering deletion!
In order to maintain invariants, the Trie must be **self-balancing**
- This guarantees logarithmic height!

Deleting from a B-tree:
1) Find and delete the key
- If the key is not in a leaf node, you need to find a replacement…

2) Next, rebalance the tree
- Is there a direct sibling node with more than minimum keys? (Is it more than ½ full)
    If so, **rotate** right/left accordingly (**Donate**, as long as it doesn't break itself)
- If not, we need to merge with either the **right or left sibling**
    But…why does this work??? Because this guarantees we will have a node that is ½ full.

Deletion from B-tree Example:

Some cases to consider…

1) If we delete from the root, we rotate the root, move up an element, then delete the root element.

2) Remove from one of the child nodes..

- Simply delete the node, and check if the **B-tree property** is maintained.

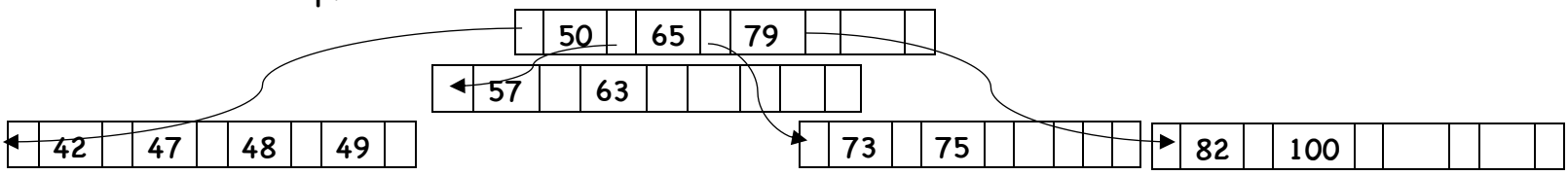
If not, perform the following algorithm..

a) Check direct neighbors, if the neighbor is more than ½ full, upgrade the rightmost/leftmost element as the root, and move the old root element into the node.

b) If none of the neighbors are more than ½ full, merge with a 1/2 full or less than ½ full node…

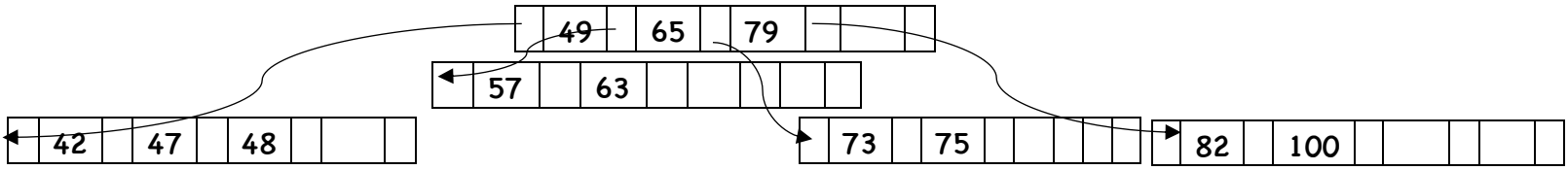
- Simply delete the element, and merge with either the left or right node, and demote the root. Decide which one to merge dependent on if the **B-tree property** will be maintained.

For Example:



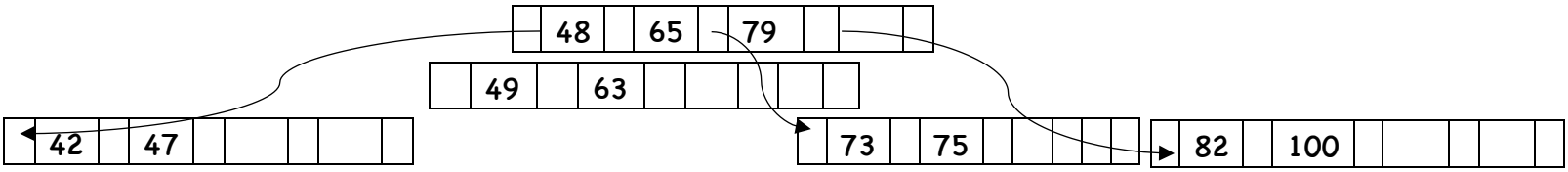
What if we wanted to delete 50?

Since 50 is a value at the root, we rotate right/left, and switch 49 with 50, and delete 50.



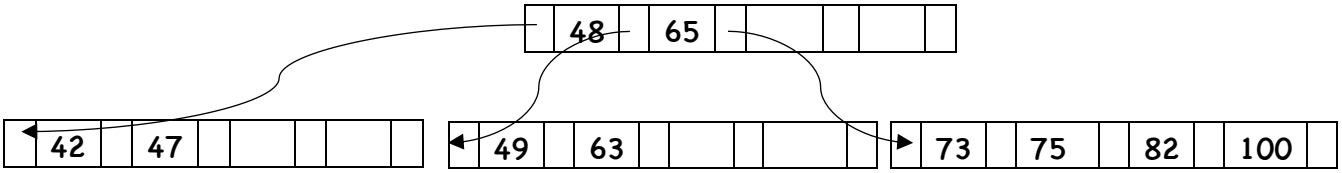
What if we deleted 57?

Well, since 57 is at a leaf node, we just delete 57, but since the node that contained 57 is not ½ full, we look at its direct siblings, and find that the left sibling is more than ½ full, so we upgrade its rightmost value as the root, and move the root value it replaced to the node that is not ½ full.



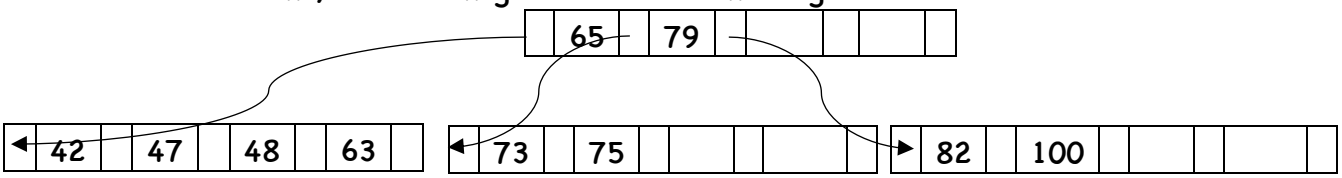
What if we deleted 82?

Well, since 82 is a leaf node, we can simply just delete it. Since this leaves that node less than ½ full, we look at its direct siblings. Notice that if the direct sibling donates one of its elements, it will also be less than ½ full. Therefore, we need to merge the two nodes. Therefore, we **demote** the root node, and merge it with the two cells, which will generate the following:



What if we deleted 49 instead?

Well, since 49 is a leaf node, we can simply just delete it. Since this leaves that node less than ½ full, we look at its direct siblings. Notice that if the direct sibling donates one of its elements, it will also be less than ½ full. Therefore, we need to merge the two nodes. Therefore, we **demote** the root node, and merge it with the two cells, which will generate the following:



Notes:

1) Leftmost node, merge with node next to it, Rightmost node, merge with node next to it.

2) Usually if choosing to merge with another node, merge with left. Doesn't matter…

Uhmmm but what if wanted to read all records???

1) How long would it take to find all disk blocks containing records?

A LONNGGGG TIMEE

2) Is there a better way?

Yes!! With B+ Trees!!

B+ Trees:

1) Maintain a copy of all the keys in the leaves of the tree.

2) Create a linked-list out of the leaf nodes of the tree.

B+ vs B Trees

1) Order

- M is the max number of children
- Elsewhere, could be the minimum number of keys

a) Min was the original notation, but it's ambiguous.

2) Where to go to follow = keys (**B+ Trees**)

- Some implementations have left link point to keys <=, and right point to keys strictly >
- Others do the opposite

Real-World Applications:

1) Storing records in a database

a) How does the DB store records?

- IBM DB2, SQL Server, etc. use B+ trees to store tables indexes

2) Other applications?

a) NTFS, ReiserFS, NSS, XFS, etc. use B+ Trees for metadata indexing.

Hashing:

Wouldn't it be Wonderful if…
We could search through a collection in $\boldsymbol{\theta(1)}$ with relatively small memory?

How about we try this:
1) Let's assume we have an array of length **"m"**
- For now, let's call it **HT**.

2) Let's also assume that we have a function, **h(x)**, that maps from our **key space** to $\{\mathbf{0}, \mathbf{1}, \mathbf{2}, \dots, \boldsymbol{m} - \mathbf{1}\}$.
- $h: \mathbb{Z} \to \{0,1,2, \dots, m-1\}$ for **integer keys**.
- Let's also assume that **h(x)** is efficient to compute!!

3) Well, it just happens that this is the basic premise of **Hash Tables!**

So, how do we SEARCH/INSERT with a Hash Map?
1) **Insert:**
$\boldsymbol{i} = \boldsymbol{h}(\boldsymbol{x})$: i refers to a key
$\boldsymbol{HT}[\boldsymbol{i}] = \boldsymbol{x}$: HT[i] represents an element!

2) **Search:**
$\boldsymbol{i} = \boldsymbol{h}(\boldsymbol{x})$: i refers to a key
$\boldsymbol{if}(\boldsymbol{HT}[\boldsymbol{i}] == \boldsymbol{x})\ \ \boldsymbol{return\ true}$; : If the element at this key is the value we're looking for, return true, otherwise return false!
$\boldsymbol{else\ return\ false}$;

3) This is a very general, simple approach to a Hash Table implementation…
- Where will we run into problems?
Well we could get into collisions!

What do we do if $h(x) == h(y)$ where x != y? Called a **Collision**
- "Different keys, x and y, hash to the same place"
Consider the following example…
1) A company has 500 employees

2) Records are stored using a HashMap with 1000 entries

3) Employee SSNs are hashed to store records in the HashMap
- Keys are SSNs, so $|keyspace| == 10^9$

4) Specifically, what keys are needed can't be known in advance
- Due to employee turnover

5) What if one employee (with SSN x) is fired and the replacement has an SSN of y?
- Can we design a hash function that guarantees h(y) does not collide with the 499 other employees' hashed SSNs? NOPE!

But, can we ever guarantee collisions will not occur?
1) Yes! If our keyspace is smaller than our HashMap!
a) If $|keyspace| \leq m$, then we can use **perfect hashing**!
    ai) I.e., **perfect hashing** is a hash function that maps every key to a distinct integer < m
    aii) Note, **perfect hashing** can also be used if n < m, and the keys to be inserted are known in advance.
        aiij) E.g. hashing the keywords of a programming language during compilation.

2) But, if $|keyspace| > m$, collisions cannot be avoided.

Handling Collisions:
1) Can we reduce the number of collisions?
a) Using a good hash function is a start!
    ai) But, what makes a good hash function??
        aij) Utilize the entire key
        aijj) Exploit differences between keys
        aijjj) Uniform distribution of hash values should be produced

Examples: Hash Functions

1) Hash List of Classmates by phone number

Bad Hash Function:

- Use the first 3 digits! Because there's likely to be more collisions, since many classmates may share similar area codes!

Better Hash Function:

- Consider the phone number as a single integer…
- Take the integer and modulo it by **m** (m is the table size)

2) Hash Words

Bad Hash Function:

- Add up all the ASCII values!!

Better Hash Function:

- Use Horner's Method to do **Modular Hashing** again!

Examples: Horner's Method

Base 10 (Decimal):

$12345: (1 \times 10^4) + (2 \times 10^3) + (3 \times 10^2) + (4 \times 10^1) + (5 \times 10^0)$

Base 2 (Binary):

$10100: (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (0 \times 2^0)$

Base 16 (Hexadecimal):

$BEEF3: (11 \times 16^4) + (14 \times 16^3) + (14 \times 16^2) + (15 \times 16^1) + (3 \times 16^0)$

ASCII Strings:

$$HELLO: ('H' \times 256^4) + ('E' \times 256^3) + ('L' \times 256^2) + ('L' \times 256^1) + ('O' \times 256^0)$$
$$= (72 \times 256^4) + (69 \times 256^3) + (76 \times 256^2) + (76 \times 256^1) + (79 \times 256^0)$$

Modular Hashing:

1) Overall, it's a good simple, general approach to implement a HashMap!

2) Here's the Basic Formula!!

$h(x) = c(x) \; mod \; m$

a) Where **c(x)** converts x into a possibly large integer (it's the base conversion that we just did!!)

b) **m** is the Table Size!!

3) Generally, we want **m** to be a prime number!!

a) Consider m = 100

b) Only the Least Significant Digits (LSDs) Matter!!

bi) I.e. h(**1**) = h(40**1**) = h(437290**1**)

Now back to Collisions!

1) Soooo we've done what we can to cut down the number of collisions, but we still need to deal with them!

2) **Collision Resolution:**

Open Addressing: "Open with Address Changing" (Closed Hashing)

Closed Addressing: "Nope, I ain't cool with my address changing" (Open Hashing)

Open Addressing:

1) I.e., if a Pigeon's Hole is taken, it has to find another one!!

2) If $h(x) == h(y) == i$

a) and x is stored i

b) If we want to insert y, we must try alternative indices!!

bi) Means that y will not be stored at $HT[h(y)]$

bij) We must select alternative in consistent and predictable ways, so that they can be located later (Gotta be a procedural, can't be random!!)

Linear Probing:
If our 1st preferred spot is not available, go to the next door until you find an empty house!

Insertion:
If we can't store a key at index i due to a collision…
1) Attempt to insert the key at $(i + 1)$, then $(i + 2)$, …, and so on…
2) $mod\ m$
3) Until open space is found!

Search:
If another key is stored at index i…
1) Check $(i + 1)$, $(i + 2)$, $(i + 3)$, … until…
    a) The key is found…
    b) An empty location is found…
    c) We circle through the buffer back to i…

Example: Linear Probing
$h(x) = x\ mod\ 11$
Insert: 14, 17, 25, 37, 34, 16, 26
Table Size is 11…
1) We want to insert 14…
14 mod 11 is 3, so we insert 14 at position 3…

0	1	2	3	4	5	6	7	8	9	10
			14							

2) We want to insert 17…
17 mod 11 is 6, so we insert 17 at position 6…

0	1	2	3	4	5	6	7	8	9	10
			14			17				

3) We want to insert 25…
25 mod 11 is 3, but we already have something at index 3, so we check if the next index has something in it. Since it doesn't, we just insert 25 at the next index…

0	1	2	3	4	5	6	7	8	9	10
			14	25		17				

4) We want to delete 37…
37 mod 11 is 4, but since we already have something at index 4, we check if the next index has something in it. Since it doesn't, we just insert 37 at the next index…

0	1	2	3	4	5	6	7	8	9	10
			14	25	37	17				

5) We want to delete 34…
34 mod 11 is 1, since nothing is at position 1, we can simply insert 34 at index 1…

0	1	2	3	4	5	6	7	8	9	10
	34		14	25	37	17				

6) We want to insert 16…
16 mod 11 is 5, but we already have something at index 5, so we check if the next index has something in it. Since it does, we just look at the next index, and check if it has something. Since it doesn't we insert 16 at index 7…

0	1	2	3	4	5	6	7	8	9	10
	34		14	25	37	17	16			

7) We want to insert 26…
26 mod 11 is 4, but we already have something at index 4, so we check if the next index has something in it. Since it does, we just look at the next index, and check if it has something. Since it does, we just look at the next index, and check if it has something. Incidentally, this one has a value as well. So, we just look at the next index, and check if it has something. Since it still does, we repeat until we find that index 8 is empty, so we insert 26 at index 8.

0	1	2	3	4	5	6	7	8	9	10
	34		14	25	37	17	16	26		

How would deletions be handled?
1) What happens if key 17 is removed??
- We would have to rehash it!

Alrighty! We just solved Collisions!!
Well you see… not quite!
1) Consider the **load factor** $\alpha = n/m$,
- Where n is the number of items in the table
- Where m is the table size
- α represents the proportion of the table that is filled.

2) As α increases, what happens to the table performance? It DEGRADES

3) Consider an empty table using a **good hash function…**
- What is the probability that a key **x** will be inserted into any one of the indices in the hash table? $P = 1/m$

4) Consider a table that has a cluster of **c** consecutive indices occupied…
- What is the probability that a key **x** will be inserted into the index directly after the cluster? $P = c/m$

Avoiding Clustering:
1) We must make sure that even **after** a collision, all of the indices of the hash table are possible for a key. (Ensure probability of keys colliding again is unlikely)

a) The probability of filled locations needs to distributed uniformly throughout the table

Double Hashing:

After a collision, instead of attempting to place the **key x** in $(i + 1)\ mod\ m$, look at $i + h2(x)\ mod\ m$

1) h2() is a second different function.

a) It should still follow the same general rules as h() to be considered good, but needs to different from h()

ai) $h(x) == h(y)$ AND $h2(x) == h2(y)$ should be very unlikely.

aij) Hence, it should be unlikely for two keys to use the same increment.

Note: We want it to be unlikely for keys to collide again!!

Calculating h2(x):

We can calculate $h2(x)$ as $[x\ mod\ (smallest\ prime\ after\ m)] + 1$

- We then calculate the offset by adding $h2(x)\ to\ h(x)$

Example: Double Hashing

Suppose $h(x) = x\ mod\ 11$

$h2(x) = (x\ mod\ 7) + 1$

Insert: 14, 17, 25, 37, 34, 16, 26

1) We want to insert 14…

14 mod 11 is 3, since nothing is at position 3, we can safely insert 14 at position 3.

0	1	2	3	4	5	6	7	8	9	10
			14							

2) We want to insert 17…

17 mod 11 is 6, since nothing is at position 6, we can safely insert 17 at position 6.

0	1	2	3	4	5	6	7	8	9	10
			14			17				

3) We want to insert 25…

25 mod 11 is 3, since there's already something at position 3, we use the second hash function, and calculate 25 mod 7 as 4 + 1 = 5, now we add h2(x) to h(x) to get the desired location of 25, as (5 + 3) = 8. We now check if there is a value at 8. Since there is not, we simply add 25 at position 8.

0	1	2	3	4	5	6	7	8	9	10
			14			17		25		

4) We want to insert 37…

37 mod 11 is 4, since nothing is at position 4, we can safely insert 17 at position 4.

0	1	2	3	4	5	6	7	8	9	10
			14	37		17		25		

5) We want to insert 34…

34 mod 11 is 1, since nothing is at position 1, we can safely insert 34 at position 1.

0	1	2	3	4	5	6	7	8	9	10
	34		14	37		17		25		

6) We want to insert 16…

16 mod 11 is 5, since nothing is at position 5, we can safely insert 16 at position 5.

0	1	2	3	4	5	6	7	8	9	10
	34		14	37	16	17		25		

7) We want to insert 26…

26 mod 11 is 4, since there's already something at position 4, we use the second hash function, and calculate 26 mod 7 as 5 + 1 = 6, now we add h2(x) to h(x) to get the desired location of 25, as (6 + 4) = 10. We now check if there is a value at 10. Since there is not, we simply add 26 at position 10.

0	1	2	3	4	5	6	7	8	9	10
	34		14	37	16	17		25		26

A Few Extra Rules for h2():

1) The second hash function cannot map to a value of 0.
- Will cause a collision!

2) You should try all indices once before trying one again!
- Try everything before we overlap!

Were either of these issues for Linear Probing?

If hash table gets filled completely, we need to rehash and resize the table.

If we have to consider deletion, we have to rehash!
- **Sentinel Key:** Indication that something used to be there, but now there's nothing.
- Have the sentinel key (placeholder) until insertion and deletion, and then replace it.

But… as $\alpha \to 1$…

1) As $n \to m$… (I.e. as the table gets more filled)
2) Both linear probing and double hashing degrade to $\theta(n)$ (Might be last place…)
    a) HOW???
        ai) Multiple collisions will occur in both schemes
        aii) Consider insertions and misses
            aiij) Both will continue until an empty index is found…
                aiijk) With few indices available, close to m probes will need to be performed.
                aiijkl) It becomes $\theta(m)$
                aiijjk) $n \to m$, so it degrades to $\theta(n)$

Essentially, you can have Constant Time, until your table gets filled up!

Open Addressing Issues:

1) We must keep a portion of the table empty in order to maintain a respectable performance
    a) For linear hashing ½ is a good rule of thumb!
        ai) It can go higher with double hashing though!

Closed Addressing:

1) I.e. If a pigeon's hole is taken, it just decides to live with a roommate.

2) Most commonly done with **Separate Chaining!**

a) Create a Linked-List of Keys (that want to live at the same location) at each index in the table…

ai) As with DLBs, **performance depends on the chain length**

aij) Chain Length is determined by α and the quality of the hash hash function. (If it's bad it'll get close to linear!)

If all of them end up in the same location, it just becomes Sequential Search"

- If it gets to a certain **threshold**, resize and rehash.

Doesn't have to be a Linked-List, could be Self-Balancing BST or something else!

In general…

Closed Addressing Hash Tables are fast and efficient for a large number of applications

Really depends…

1) **Strict Memory: Open Addressing**
2) **Memory Available: Closed Addressing**

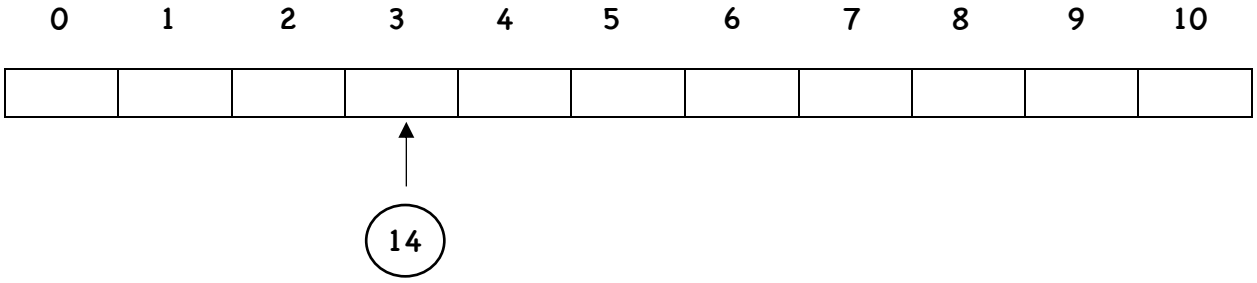
Example: Separate Chaining

Suppose $h(x) = x \, mod \, 11$

Insert: 14, 17, 25, 37, 34, 16, 26

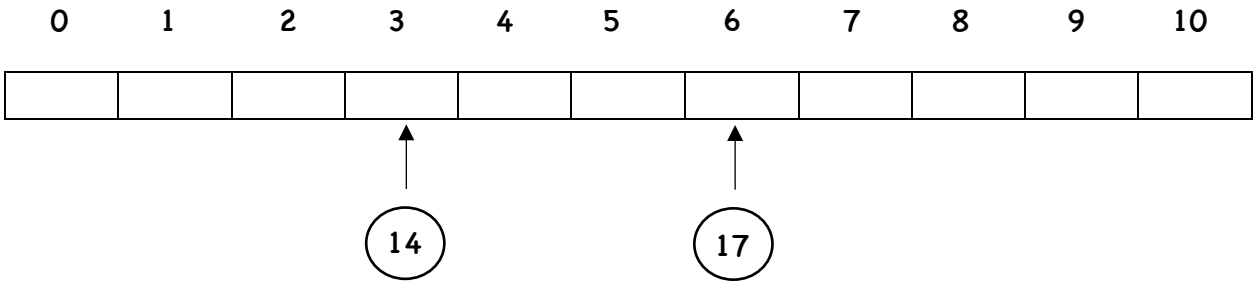
1) Want to insert 14…

14 mod 11 is 3, since nothing is at position 3, we can safely insert 14 as the first link to index 3.



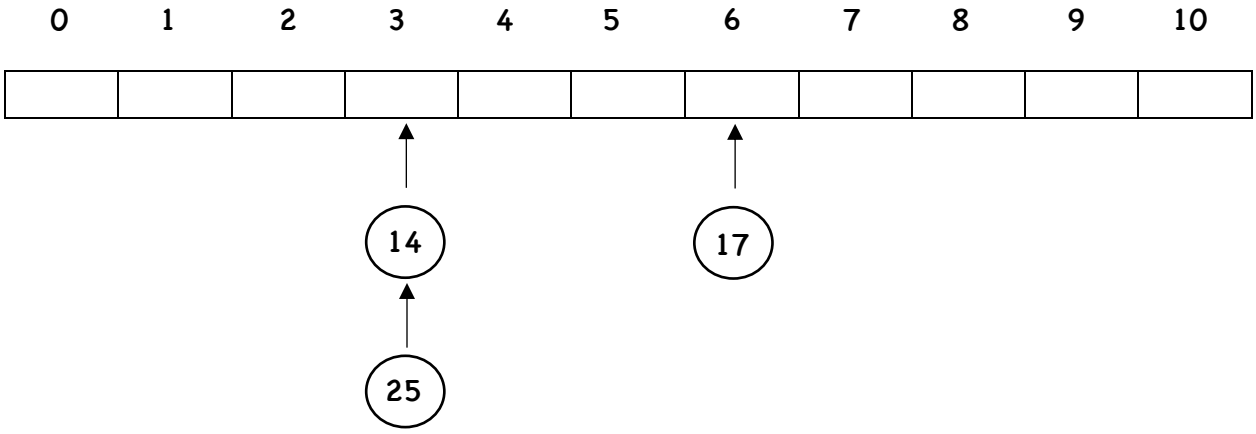
2) Want to insert 17…

17 mod 11 is 6, since nothing is at position 6, we can safely insert 17 as the first link to index 6.



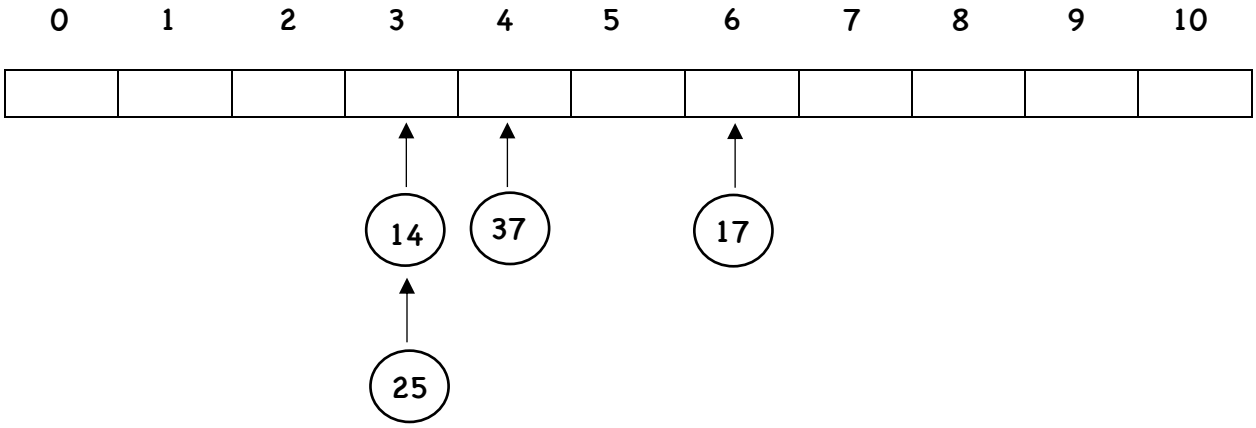
3) Want to insert 25…

25 mod 11 is 3, since there's already a link is at position 3, we can just insert 25 as the second link to index 3.



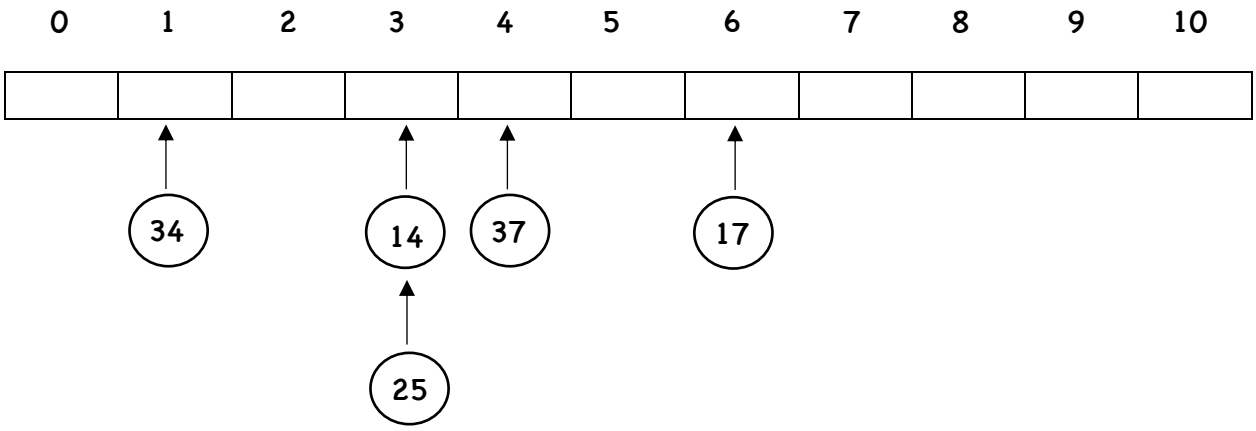
4) Want to insert 37…

37 mod 11 is 4, since nothing is at position 4, we can safely insert 37 as the first link to index 4.



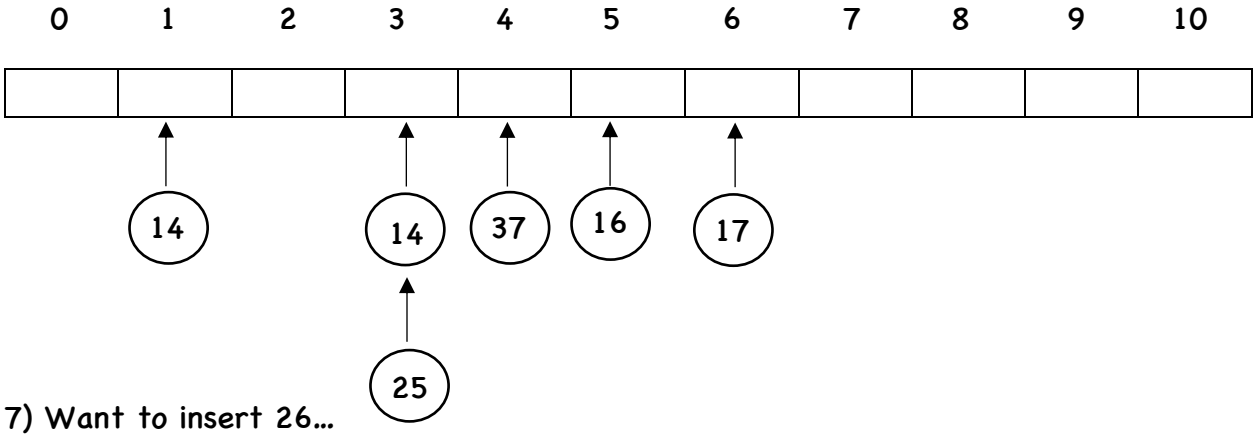
5) Want to insert 34…

34 mod 11 is 1, since nothing is at position 1, we can safely insert 34 as the first link to index 1.



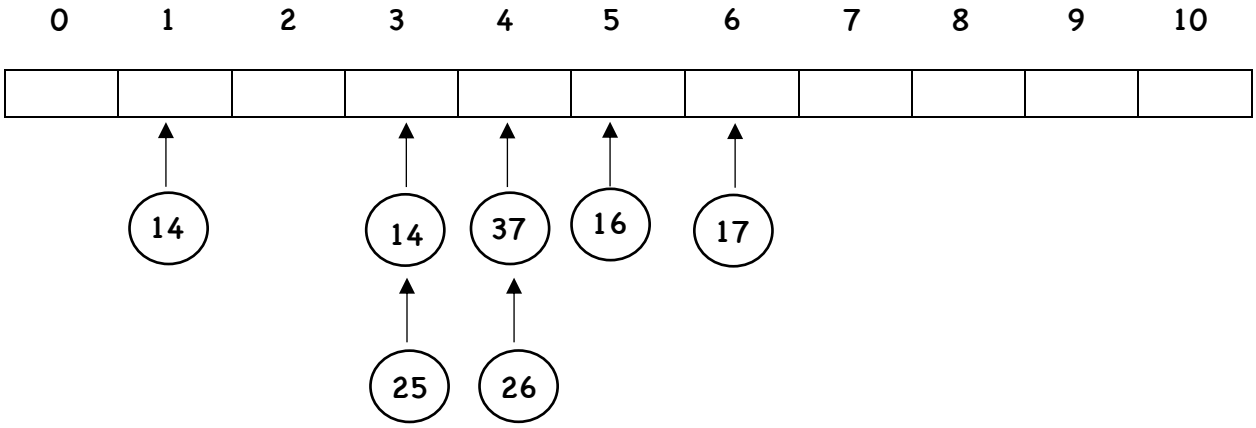
6) Want to insert 16…

16 mod 11 is 5, since nothing is at position 5, we can safely insert 16 as the first link to index 5.



7) Want to insert 26…

26 mod 11 is 4, since there's already a link is at position 4, we can just insert 26 as the second link to index 4.



Compression:

What is Compression?

1) Representing the "**same**" data using less storage space!
    a) We can get more use out of a disk of a given size!
    b) We can get more use of memory!
        bi) E.g. We can free up more memory by compressing inactive sections!
            bij) Faster than paging!
            bijj) Built into OSX Mavericks and later!
    c) We can reduce the amount of data transmitted!
        ci) Faster file transfers!
        cii) Cut power usage on mobile devices!
2) There are two main approaches to compression!

Lossy Compression:

- It doesn't look as good, but it's good ENOUGH!

Basic Model:

$$D \rightarrow [COMPRESS] \rightarrow C \rightarrow [EXPAND] \rightarrow D' \neq D$$

1) Might not be same original data, but we can fill in the blanks!

2) Information is **permanently lost** in Compression.
- MP3, H264, JPEG

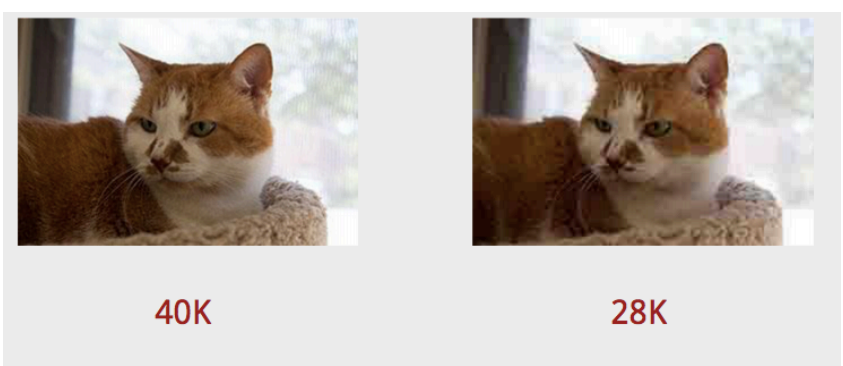
3) With audio/video files, this typically isn't a huge problem as human users might not be able to perceive the difference!

Example: Lossy Compression

1) **MP3**

"Cuts out" portions of audio that are considered beyond what most people are capable of hearing.

2) **JPEG**



Lossless Compression:

Basic Model:

$$D \rightarrow [COMPRESS] \rightarrow C \rightarrow [EXPAND] \rightarrow D = D$$

1) Input can be recovered from compressed data exactly!

- zip files, FLAC

Huffman Compression:

1) Works on arbitrary bit strings, but pretty easily explained using characters!

2) Consider the ASCII character set…

a) Essentially blocks of codes

ai) In general, to fit R potential characters in a block, you need **lg R** bits of storage per block!

aij) Consequently, **n** bit storage blocks represent 2^n characters!

aii) Each 8 bit code block represents one of 256 possible characters in extended ASCII!

aiii) Easy to encode/decode!

Considerations for Compressing ASCII:

1) What if we used **variable length** codewords instead of the constant 8?

- Could we store the same information in less space?

a) Different characters are represented using codes of different bit lengths!

b) If all characters in the alphabet have the same usage frequency, we can't beat block storage!!

bi) On a character by character basis…

c) But what about different usage frequencies between characters?

ci) In English, R, S, T, L, N, E are used much more than **Q** or **X**

- So, Q or X will be encoded differently!

Variable Length Encoding:

Decoding was easy for Block Codes!

1) Grab the next 8 bits in the bitstring!

2) How can we decode a bitstring that is made up of variable length code words?

BAD Example of Variable Length Encoding:

1	A
00	T
01	K
001	U
100	R
101	C
10101	N

Could be 1 00 00 1 101 01 (A T T A C K) or 100 001 10101 (R U N)
- There's no delimiter!

Variable Length Encoding for Lossless Compression:
Codes must be **prefix-free**!
1) No code can be a prefix of any other in the scheme
2) Using this, we can achieve compression by…
    a) Using **fewer bits** to represent **more common characters**!
    b) Using **longer codes** to represent **less common characters**!

How can we create these prefix-free codes?
HUFFMAN ENCODING!

Generating Huffman Codes:
1) Assume we have **K** characters that are used in the file to be compressed and each has a weight (its frequency of use)

2) Create a forest (set of trees), of **K** single-node trees, one for each character, with the single node storing that character's weight.

3) while | F | > 1:
    a) Select T1, T2 $\in$ F that have the smallest weights in F.
    b) Create a new tree node N whose weight is the sum of T1 and T2's weights!
    c) Add T1 and T2 as children (subtrees) of N
    d) Remove T1 and T2 from F.
    e) Add the new tree rooted by N to F!

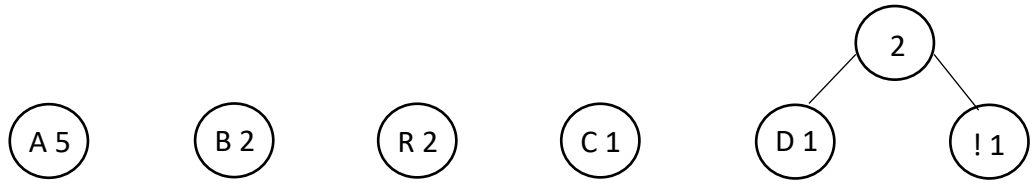
Example: Huffman Encoding: ABRACADABRA!
Let's construct a Huffman Trie with the String **ABRACADABRA!**

Let's calculate the Frequencies first…
1) There are 5 A's, so A will have a weight of 5…
2) There are 2 B's, so B will have a weight of 2…
3) There are 2 R's, so R will have a weight of 2…
4) There's only 1 C, so C will have a weight of 1…
5) There's only 1 D, so D will have a weight of 1…
6) There's only 1 "!", so "!" will have a weight of 1…

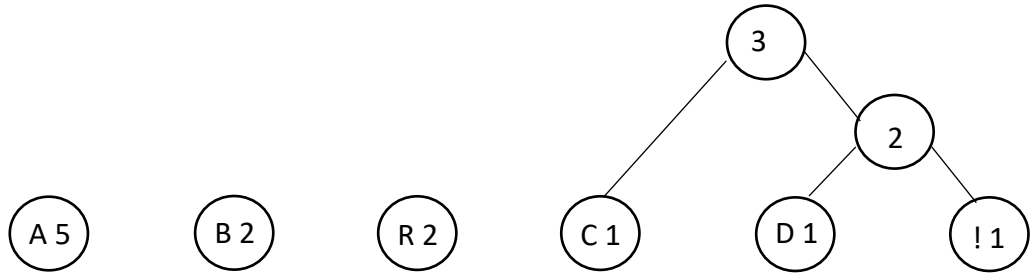
Put the values from left to right, first based on frequencies, then if they have equal frequencies, by the order in which they appear in the pattern…

So, without construction of the forest, the "tree" currently looks like…

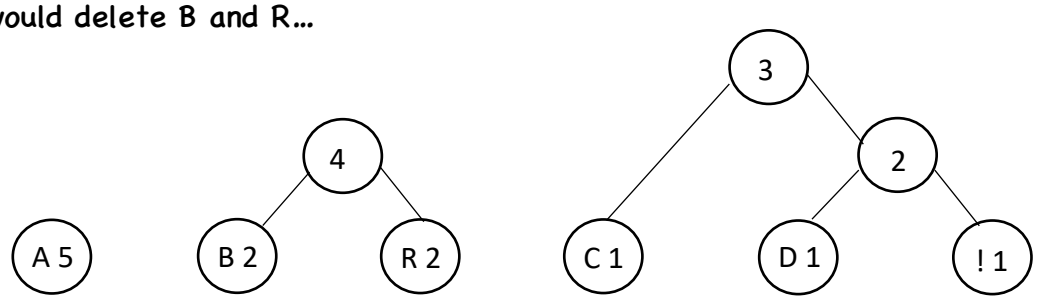
Now, we can start building the forest…
1) We're gonna look for the 2 smallest Trees, which are currently D and !, so we combine them together along with their weights, then we remove **both D and !**



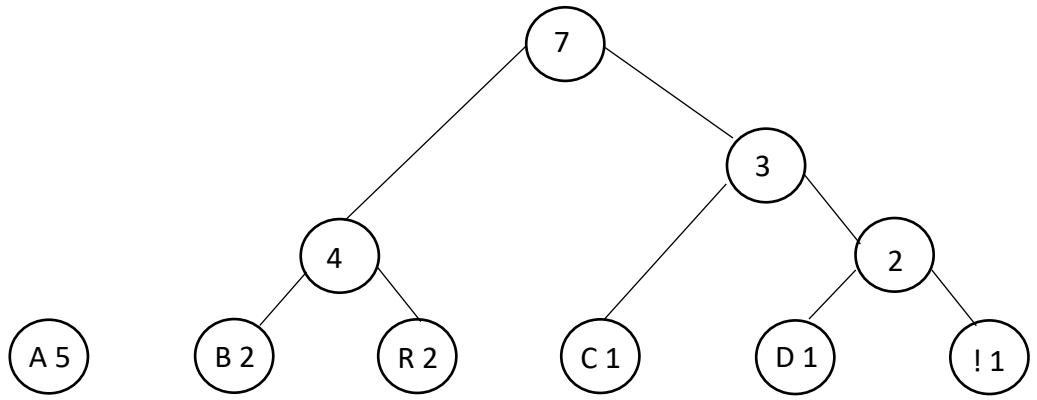
2) We're then gonna look for the next smallest from right to left, which would currently be 2, and C, so we merge them together to get a weight of 3, and we would delete 2 and C…



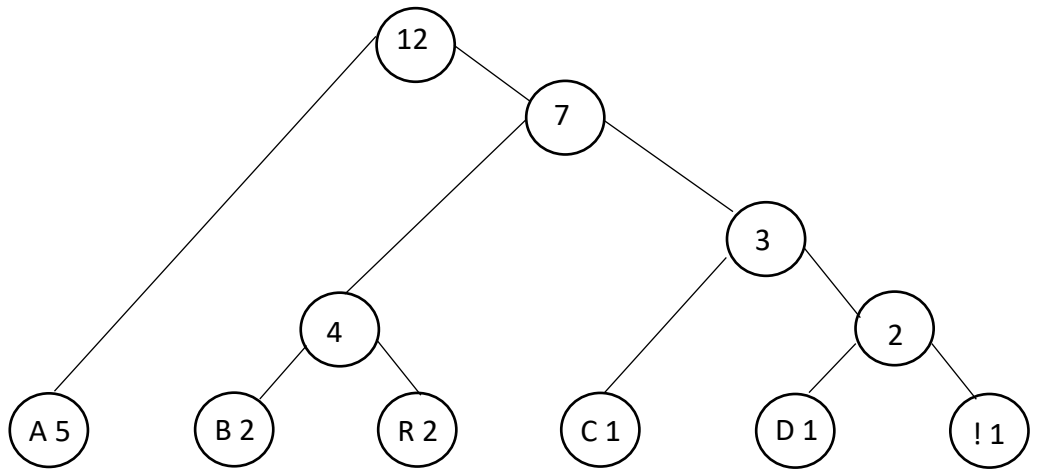
3) We're then gonna look for the next smallest from right to left, which would currently be B, and R, so we merge them together to get a weight of 4, and we would delete B and R…



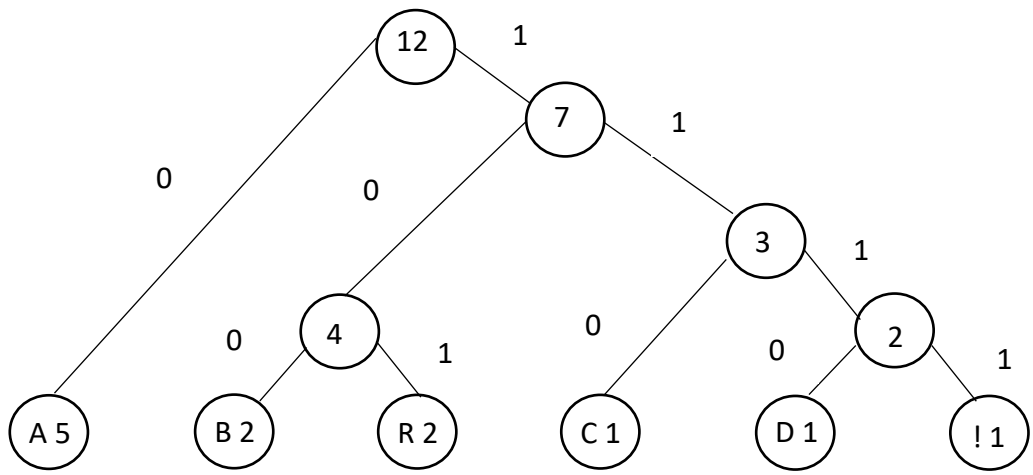
4) We're then gonna look for the next smallest from right to left, which would currently be 3, and 4, so we merge them together to get a weight of 7, and we would delete 3 and 4…



5) We're then gonna look for the next smallest from right to left, which would currently be 5, and 7, so we merge them together to get a weight of 12, and we would delete 5 and 7…



6) Now that we've built the Huffman trie, we can encode the path, if we go left encode it as a 0, and if we go right encode it as a 1…



7) To get the compressed bitstring, we combine the encoded paths, and attempt to form the bitstring…

0 100 101 0 110 0 1110 0 100 101 0 1111

A B R A C A D A B R A !

Implementation Concerns:

1) We need to efficiently be able to select the lowest weight trees to merge when constructing the trie!

a) We can accomplish this by using a Priority Queue!

ai) minPQ – lowest weight first! (add & remove: $\theta(\log(n))$)

2) We need to be able to read and write bitstrings!

a) Unless we pick multiples of 8 bits (bytes!) for our codewords, we will need to read and write fractions of bytes for our codewords!

ai) We're not actually going to do I/O on fraction of bytes!

aii) We'll maintain a **buffer of bytes** and perform bit processing on this buffer!

aiij) Pretend we're only looking at one bit at a time!

aiii) **BinaryStdIn.java and BinaryStdOut.java!!**

Binary I/O:

```
private static void writeBit(boolean bit) {
        // add bit to buffer
        buffer <<= 1;
        if (bit) buffer |= 1;
        // if buffer is full (8 bits), write out as a single byte
        N++;
        if (N == 8) clearBuffer();
}
```

1) buffer <<= 1: Left Shift the Bits by 1 and make room for the new bit!
2) if(bit) buffer |= 1 (Incorporate bit and LSD made into 1 if bit == true)
3) if(N == 8): If we have a byte, write to disk and clear the buffer!

Example: Binary I/O Write

Consider the following sequence of operations:

1) **writeBit(true)**
2) **writeBit(false)**
3) **writeBit(true)**
4) **writeBit(false)**
5) **writeBit(false)**
6) **writeBit(false)**
7) **writeBit(false)**
8) **writeBit(true)**

After the first operation…

writeBit(true) **buffer:** ???????0 → ???????1

N: **1**

After the next operation…

writeBit(false) **buffer:** ???????1 → ??????01

N: **2**

After the next operation…

writeBit(true) **buffer:** ??????01 → ?????011

N: **3**

After the next operation…

writeBit(false) **buffer:** ?????011 → ?????0011

N: **4**

After the next operation…

writeBit(false) **buffer:** ????0011 → ???00011

N: 5

After the next operation…

writeBit(false) **buffer:** ???00011 → ??000011

N: 6

After the next operation…

writeBit(false) **buffer:** ??000011 → ?0000011

N: 7

After the next operation…

writeBit(true) **buffer:** ?0000011 → 00000111

N: 8

Since buffer is full, reset!!

Read is the opposite! Look at the MSD instead!

Further Implementation Concerns:

1) To encode/decode, we'll need to **read in characters** and **output codes/read in codes** and **output characters**!

a) Sounds like we'll need a **Symbol Table!**

ai) What implementation would be best?

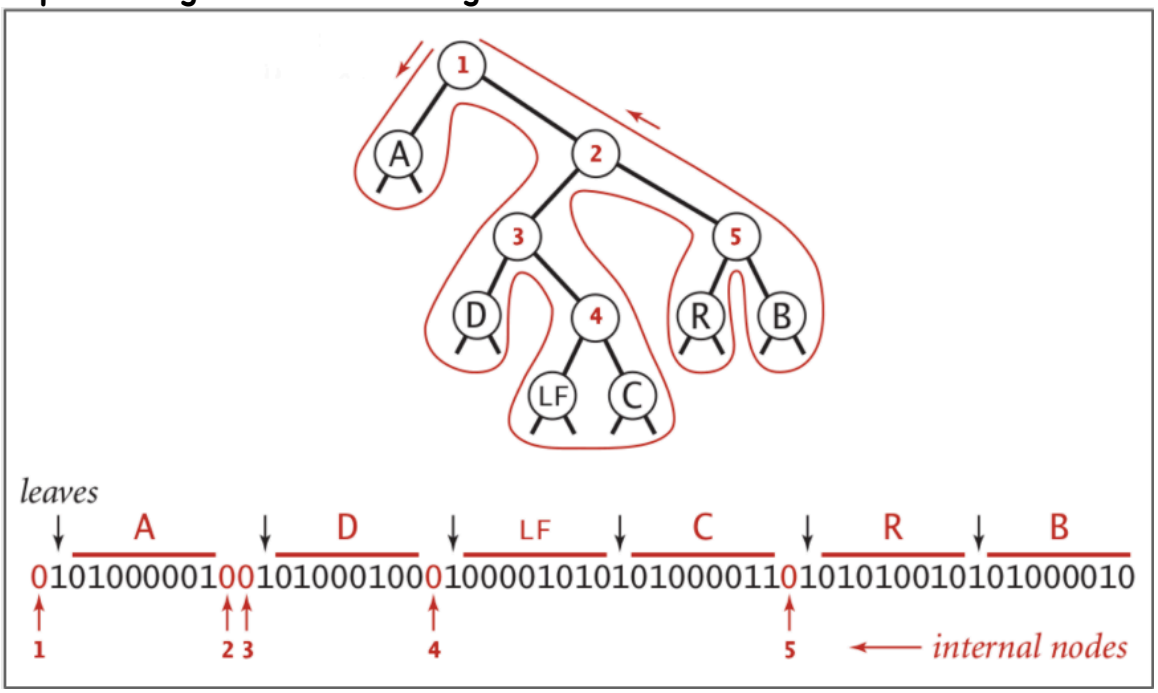
aij) Same for encoding and decoding? (Encoding: HT, Decoding: Trie)

aijk) No, we need something else to compress the data!

b) **This means we need access to the Trie to expand a compressed file!**

bi) Encode Trie and Put in file, do a traversal, land on a leaf node, add path to array at that position!

Representing Tries as BitStrings:



Traversing the trie…

1) Write a 0 if it's either an **Internal Node** or the **Root Node**!

2) Write a 1 if it's a Leaf followed by a character stored at the leaf…

Perform a Pre-order Traversal (Go left, then right….)

Example:

1) To get to A, we first start at the root… (so write a 0), then go left to the leaf (so write a 1), then we write the ASCII value for A in binary…

2) Now, imagine that we've encoded the root into our path, and we're simply trying to continue traversing through the tree. In other words, pretend 0 is already encoded. Now, we want to traverse leftwards, so we go to internal nodes 2 and 3, before arriving at leaf D. The path for this following the root would be 001 (00 for the internal nodes traversed, and 1 for the leaf node), then we follow this path with the ASCII value for B in binary…

3) Now, imagine that we've encoded the previous path into our path, and we're simply trying to continue traversing through the tree. In other words, pretend 000 is already encoded. Now, we want to traverse rightwards from the internal node 3, so we go to internal node 3, and traverse left to leaf node LF. The path for this following the root would be 1 (1 for the leaf node), then we follow this path with the ASCII value for LF in binary…

4) Now, imagine that we've encoded the previous path into our path, and we're simply trying to continue traversing through the tree. In other words, pretend 000 is already encoded. Now, we want to traverse rightwards from the internal node 3, so we go to internal node 3, and traverse right to leaf node C. The path for this following the root would be 1 (1 for the leaf node), then we follow this path with the ASCII value for C in binary…

5) Now, that we've traversed through all of the internal nodes within 2, as well as its leaf nodes. Imagine that we've encoded the previous path until internal node 2 into our path, and we're simply trying to continue traversing through the tree. In other words, pretend 00 is already encoded. Now, we want to traverse rightwards from the internal node 2, so we go to internal node 5, and traverse left to leaf node R. The path for this following the root would be 01 (0 to get to 5, and 1 for the leaf node), then we follow this path with the ASCII value for R in binary…

6) Now, imagine that we've encoded the previous path into our path, and we're simply trying to continue traversing through the tree. In other words, pretend 000 is already encoded. Now, we want to traverse rightwards from the internal node 5, so we go to internal node 5, and traverse right to leaf node B. The path for this following the root would be 1 (1 for the leaf node), then we follow this path with the ASCII value for LF in binary…

Binary I/O Trie Encoding:

```
private static void writeTrie(Node x){
    if (x.isLeaf()) {
        BinaryStdOut.write(true);
        BinaryStdOut.write(x.ch);
        return;
    }
    BinaryStdOut.write(false);
    writeTrie(x.left);
    writeTrie(x.right);
}

private static Node readTrie() {
    if (BinaryStdIn.readBoolean())
        return new Node(BinaryStdIn.readChar(), 0, null, null);
    return new Node('\0', 0, readTrie(), readTrie());
}
```

1) if(x.isLeaf()): if we're at a leaf node, write out a 1 (**BinaryStdOut.write(true)**),and write the string's value encoded in Binary(**BinaryStdOut.write(x.ch)**)...

2) Otherwise, we just indicate that we're just going through either **internal nodes** or the **root node** (**BinaryStdOut.write(false)**). Then recursively go through the left, then the right (**Because it's PREORDER (>w<)**)

In regard to reading the Trie... (I.e. decoding...)
1) First determine whether we're traversing through a leaf node or an internal node or root node... (**BinaryStdIn.readBoolean()**) -> **True** leaf node...

2) If it is a Leaf Node, Read in the String...
(**return new Node(BinaryStdIn.readChar(), 0, null, null));**

3) Otherwise, recursively look at the left and right children...
(**return new Node('\0', 0, readTrie(), readTrie());**

Huffman Pseudocode:

Encoding Approach:

1) Read input
2) Compute Frequencies
- Tells you when to **STOP READING**
3) Build trie/codeword table (via **Huffman Encoding!**)
4) Write out trie as a BitString to Compressed File!
5) Write out trie as character count of input!
6) Use table to write out the codeword for each input character!
- Write out the Character Count followed by the codeword outputted to the Compressed File!

Decoding Approach:

1) Read Trie (Reconstruct Trie)
2) Read the Character Count
3) Use the Trie to decode bitstring of Compressed File!
- One at a time, outputting the character we get at a leaf!!

How do we determine character frequencies?

Option 1: Preprocess the file to be compressed

1) **Upside:** Ensures that Huffman's algorithm will produce the best output for the given file!

2) **Downsides:**

a) Requires two passes over the input, **one** to analyze frequencies/build the trie/build the code lookup table, and **another** to compress the file!
- This is mad slow!
- Have to take whole thing, and wait until we know the frequencies, **AND THEN SEND IT**!

b) Trie must be stored with the Compressed File, reducing the quality of compression!
- Puts us over the edge! (Reduces how much compression we actually get out!)

bi) This especially hurts small files!

bii) Generally, large files are more amenable to Huffman Compression!
- Need a bit bigger, so the Benefits will outweigh Costs!

biij) Just because a file is large, it does not mean that it will compress well!
- Don't merge files! **DOES NOT GUARANTEE BETTER COMPRESSION!**

Option 2: Use a Static Trie

1) Analyze multiple sample file, build a single tree that will be used for all compressions and expansions!

- Files hurt each other!

2) Saves trie storage overhead

- By in-large will lose, since Frequencies will not get stored as if we read from one file instead!

3) In general, a pretty bad approach!

a) Different character frequency characteristics of different files means that a code set/trie that works well for one file could work very poorly for another!

- No one is **BETTER OFF** :(

ai) Could even cause an increase in file size after "**compression**"

Bad Trie -> Bad Bad :(

Option 3: Adaptive Huffman Encoding

1) Single pass over the data to construct the codes and compress a file with no background knowledge of the source distribution!

- Constructing codes in such a way where we can get the **ENTIRE SOURCE**!
- Learning distribution as we Compress!

2) Not gonna be covered though!!

So, ermm, how good is Huffman Compression?

1) ASCII requires $8m$ bits to store m characters!

2) For a file containing c different characters…

a) Given Huffman codes $\{h_0, h_1, h_2, \dots, h_{(c-1)}\}$

b) and **frequencies** $\{f_0, f_1, f_2, \dots, f_{(c-1)}\}$

c) Sum from 0 to c-1: $|h_1| * f_i$

- I.e. $\sum_{i=0}^{c-1} |h_i| * f_i$ **where** $|h_i|$ is the size of the representation, and f_i is the number of times it appears!

3) Total storage depends on the **differences in frequencies**!

a) The **bigger the differences**, the **better the potential for compression**!

- Wider Variance -> Better potential for compression!

Huffman is optimal for **character-by-character prefix-free encodings**!

That seems like a significant caveat!

1) Where does Huffman fall short?

- If all have the same frequencies and there are repeated patterns!

    a) What about repeated patterns of multiple characters?

        ai) Consider a file containing…

            aij) 1000 A's

            aijj) 1000 B's

            aijjj) …

            aijjjj) 1000 of every ASCII character!!

    b) Will this compress at all with Huffman encoding? **NOPE**

    c) But it seems like it should be compressible…

Run Length Encoding:

We could just represent the previously mentioned string as:

1) 1000A1000B1000C, etc.

    a) Assuming we use 10 bits to represent the number of repeats, and 8 bits to represent the character…

        ai) 4608 bits (18 * 256) bits needed to store run length encoded file!

        aii) vs. 2048000 bits for input file!

        aiii) **WOAH HUGE SAVINGS!**

2) This incredible compression performance is based on a **VERY SPECIFIC SCENARIO**

    a) Run length encoding is not generally effective for **most** files, as they often lack long runs of repeated characters!

What else ca we do to compress files?

LZW!!!

Patterns are compressible, but we need a general approach!!

1) Huffman used **variable-length codewords** to represent **fixed-length portions of the input**…

a) Let's try another approach that uses **fixed-length (same size) codewords** to represent **variable-length portions of the input** (Represent multiple characters)

2) **Idea:** The more characters that can be represented in a single codeword, the better the compression!

a) Consider **"the"**: 24 bits in ASCII

b) Representing **"the"** with a single 12 bit codeword **cuts the used space in half**!

bi) Similarly, representing longer strings with a 12 bit codeword would mean **even better savings**!!

We're building up the codebook while compressing, such that expansion can decipher the codebook during compression!

How do we know that "the" will be in our file?

1) We need to avoid the same problems that were raised with the use of a static trie for Huffman Encoding!

2) So, use an **adaptive algorithm** and build up our patterns and codewords as we go through the file!

LZW Compression:

1) Initialize codebook to all single characters **(Maps to itself)**

- E.g., **Character maps to its ASCII value**
- **Right now, the codebook SUCKS**

2) While **!EOF**:

a) Match the longest prefix in the codebook!

b) Output the codeword!

- Output **n** bit codeword that matches that prefix!!

c) Take this longest prefix, add the next character in the file, and add the result to the dictionary with a new codeword!

Methodically adding in a way such that expansion can mimic this process!!

Example: LZW Compression

Suppose we want to **Compress** using 12 bit codewords:

1) **TOBEORNOTTOBEORTOBEORNOT**

We're gradually building up patterns as we compress…

1) We start out by looking at the first character **T**…

- We search for the corresponding value in the codebook, and output the corresponding value and add T concatenated with the next character O, I.e., TO, as the next index in the codebook, 256, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256

2) Now we try adding **O**…

- We search for the corresponding value in the codebook, and output the corresponding value and add O concatenated with the next character B, I.e., OB, as the next index in the codebook, 257, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257

3) Now we try adding B…

- We search for the corresponding value in the codebook, and output the corresponding value and add B concatenated with the next character E, I.e., BE, as the next index in the codebook, 258, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258

4) Now we try adding E…

- We search for the corresponding value in the codebook, and output the corresponding value and add E concatenated with the next character O, I.e., EO, as the next index in the codebook, 259, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259

5) Now we try adding O…

- We search for the corresponding value in the codebook, and output the corresponding value and add O concatenated with the next character R, I.e., OR, as the next index in the codebook, 260, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260

6) Now we try adding R…

- We search for the corresponding value in the codebook, and output the corresponding value and add R concatenated with the next character N, I.e., RN, as the next index in the codebook, 261, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261

7) Now we try adding N…

- We search for the corresponding value in the codebook, and output the corresponding value and add N concatenated with the next character O, I.e., NO, as the next index in the codebook, 262, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262

8) Now we try adding O…
- We search for the corresponding value in the codebook, and output the corresponding value and add O concatenated with the next character T, I.e., OT, as the next index in the codebook, 263, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263

9) Now we try adding T…
- We search for the corresponding value in the codebook, and output the corresponding value and add T concatenated with the next character T, I.e., TT, as the next index in the codebook, 264, in case the pattern shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264

10) Now we try adding TO, since it's a pattern in our codebook! So, we output the corresponding value, 256, and add TO concatenated with the next character B, I.e., TOB, as the next index in the codebook, 265, in case the shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB: 265

11) Now we try adding BE, since it's a pattern in our codebook! So, we output the corresponding value, 258, and add BE concatenated with the next character O, I.e., BEO, as the next index in the codebook, 266, in case the shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB:265
BE	258	BEO:266

12) Now we try adding OR, since it's a pattern in our codebook! So, we output the corresponding value, 260, and add OR concatenated with the next character T, I.e., ORT, as the next index in the codebook, 267, in case the shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB:265
BE	258	BEO:266
OR	260	ORT:267

13) Now we try adding TOB, since it's a pattern in our codebook! So, we output the corresponding value, 265, and add TOB concatenated with the next character E, I.e., TOBE, as the next index in the codebook, 265, in case the shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB:265
BE	258	BEO:266
OR	260	ORT:267
TOB	265	TOBE:268

14) Now we try adding EO, since it's a pattern in our codebook! So, we output the corresponding value, 259, and add EO concatenated with the next character R, I.e., EOR, as the next index in the codebook, 269, in case the shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB:265
BE	258	BEO:266
OR	260	ORT:267
TOB	265	TOBE:268
EO	259	EOR:269

15) Now we try adding RN, since it's a pattern in our codebook! So, we output the corresponding value, 261, and add RN concatenated with the next character O, I.e., RNO, as the next index in the codebook, 270, in case the shows up later.

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB:265
BE	258	BEO:266
OR	260	ORT:267
TOB	265	TOBE:268
EO	259	EOR:269
RN	261	RNO:270

16) Now we try adding OT, since it's a pattern in our codebook! So, we output the corresponding value, 263, but since there are no more lines to go through in the file, we just don't add anything!

Cur	Output	Add
T	84	TO:256
O	79	OB:257
B	66	BE:258
E	69	EO:259
O	79	OR:260
R	82	RN:261
N	78	NO:262
O	79	OT:263
T	84	TT:264
TO	256	TOB:265
BE	258	BEO:266
OR	260	ORT:267
TOB	265	TOBE:268
EO	259	EOR:269
RN	261	RNO:270
OT	263	--

LZW Expansion:

1) Initialize codebook to all single characters

- E.g., **ASCII value maps to its character** (Same as Compression!)

2) **While !EOF:**

a) Read the next codeword from the file!

b) Lookup the corresponding pattern in the codebook!

c) Output that pattern!

d) Add the previous pattern & the first character of the current pattern to the codebook!

- What was my pattern one step before?
- **Note: This means no codebook addition after the first pattern output!**
- Don't add anything on the first iteration ;)

Example: LZW Expansion

1) Read the first codeword from the file! Don't add anything on this iteration, but output the corresponding pattern, which is T!

Cur	Output	Add
84	T	--

2) Read the next codeword from the file! Output the corresponding pattern, which is O! Concatenate the previous pattern, T, with the first character of the current pattern, which is O, generating TO, which is added to the codebook, along with the next index, 256!

Cur	Output	Add
84	T	--
79	O	256:TO

3) Read the next codeword from the file! Output the corresponding pattern, which is B! Concatenate the previous pattern, O, with the first character of the current pattern, which is B, generating OB, which is added to the codebook, along with the next index, 257!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB

4) Read the next codeword from the file! Output the corresponding pattern, which is E! Concatenate the previous pattern, B, with the first character of the current pattern, which is E, generating BE, which is added to the codebook, along with the next index, 258!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE

5) Read the next codeword from the file! Output the corresponding pattern, which is O! Concatenate the previous pattern, E, with the first character of the current pattern, which is O, generating EO, which is added to the codebook, along with the next index, 259!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO

6) Read the next codeword from the file! Output the corresponding pattern, which is R! Concatenate the previous pattern, O, with the first character of the current pattern, which is R, generating OR, which is added to the codebook, along with the next index, 260!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR

7) Read the next codeword from the file! Output the corresponding pattern, which is N! Concatenate the previous pattern, R, with the first character of the current pattern, which is N, generating RN, which is added to the codebook, along with the next index, 261!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN

8) Read the next codeword from the file! Output the corresponding pattern, which is O! Concatenate the previous pattern, N, with the first character of the current pattern, which is O, generating NO, which is added to the codebook, along with the next index, 262!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO

9) Read the next codeword from the file! Output the corresponding pattern, which is T! Concatenate the previous pattern, O, with the first character of the current pattern, which is T, generating OT, which is added to the codebook, along with the next index, 263!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT

10) Read the next codeword from the file! It turns out that we already have the codeword **256** in the file, so we just output the corresponding value **TO**, and then concatenate the previous pattern, T, with the first character of the current pattern T, generating **TT**, which is added to the codebook, along with the next index 264!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT

11) Read the next codeword from the file! It turns out that we already have the codeword **258** in the file, so we just output the corresponding value BE, and then concatenate the previous pattern, TO, with the first character of the current pattern B, generating **TOB**, which is added to the codebook, along with the next index 265!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT
258	BE	265:TOB

12) Read the next codeword from the file! It turns out that we already have the codeword **260** in the file, so we just output the corresponding value OR, and then concatenate the previous pattern, BE, with the first character of the current pattern O, generating **BEO**, which is added to the codebook, along with the next index 266!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT
258	BE	265:TOB
260	OR	266:BEO

13) Read the next codeword from the file! It turns out that we already have the codeword **265** in the file, so we just output the corresponding value TOB, and then concatenate the previous pattern, OR, with the first character of the current pattern T, generating **ORT**, which is added to the codebook, along with the next index 267!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT
258	BE	265:TOB
260	OR	266:BEO
265	TOB	267:ORT

14) Read the next codeword from the file! It turns out that we already have the codeword **259** in the file, so we just output the corresponding value EO, and then concatenate the previous pattern, TOB, with the first character of the current pattern E, generating **TOBE**, which is added to the codebook, along with the next index 268!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT
258	BE	265:TOB
260	OR	266:BEO
265	TOB	267:ORT
259	EO	268:TOBE

15) Read the next codeword from the file! It turns out that we already have the codeword **261** in the file, so we just output the corresponding value RN, and then concatenate the previous pattern, EO, with the first character of the current pattern R, generating **EOR**, which is added to the codebook, along with the next index 269!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT
258	BE	265:TOB
260	OR	266:BEO
265	TOB	267:ORT
259	EO	268:TOBE
261	RN	269:EOR

16) Read the next codeword from the file! It turns out that we already have the codeword **263** in the file, so we just output the corresponding value OT, and then concatenate the previous pattern, RN, with the first character of the current pattern O, generating **RNO**, which is added to the codebook, along with the next index 270!

Cur	Output	Add
84	T	--
79	O	256:TO
66	B	257:OB
69	E	258:BE
79	O	259:EO
82	R	260:OR
78	N	261:RN
79	O	262:NO
84	T	263:OT
256	TO	264:TT
258	BE	265:TOB
260	OR	266:BEO
265	TOB	267:ORT
259	EO	268:TOBE
261	RN	269:EOR
263	OT	270:RNO

How does this work out?

Both compression and expansion construct the same codebook!

1) Compression stores character strings -> the codeword

- We're building a ST mapping every string that represents a pattern to an n bit codeword!

2) Expansion stores codeword -> the character string

- Building up a inverse ST!

3) They contain the same pairs in the same order!

    a) Hence, the codebook doesn't need to be stored with the compressed file, saving space!

Just one tiny little issue to sort out…

Expansion can sometimes be a step ahead of compression…

1) If, during compression, the (pattern, codeword) that was just added to the dictionary is immediately used in the next step, the expansion algorithm will not yet know the codeword!

2) This is easily detected and dealt with, however!

3) Take the previous pattern, and add to its own first character.

Example: LZW Corner Case

Compression

Suppose we want to compress, using 12 bit codewords: **AAAAAA**

1) Now we try adding A…

- We search for the corresponding value in the codebook, and output the corresponding value and add A concatenated with the next character A, I.e., AA, as the next index in the codebook, 256, in case the pattern shows up later.

Cur	Output	Add
A	65	AA:256

2) Now we try adding AA, since it a value in our codebook! So, we output the corresponding value, 256, and we add **AA** concatenated with the next character, **A**. I.e., AAA as the next index in the codebook, 257, in case the pattern shows up later!

Cur	Output	Add
A	65	AA:256
AA	257	AAA:257

3) Now we try adding AAA, since it's a pattern in our codebook! So, we output the corresponding value, 257, but since there are no more lines to go through in the file, we just don't add anything!

Cur	Output	Add
A	65	AA:256
AA	257	AAA:257
AAA	257	--

Expansion:

1) Read the first codeword from the file! Don't add anything on this iteration, but output the corresponding pattern, which is A!

Cur	Output	Add
65	A	--

2) Read the next codeword from the file! Output the corresponding pattern, which is AA! Concatenate the previous pattern, A, with the first character of the current pattern, which is A, generating AA, which is added to the codebook, along with the next index, 256!

Cur	Output	Add
65	A	--
256	AA	256:AA

3) WE HIT THE SPECIAL CASE. Read the previous pattern, AA, and add to its own first character **A**, and output **AAA**, and add **AAA** with the new index, 257!

Cur	Output	Add
65	A	--
256	AA	256:AA
257	AAA	257:AAA

LZW Implementation Concerns: Codebook

1) How to represent and store during…

- **Compression**
- **Expansion**

We need a ST for each one (LZW uses TST)

2) **Considerations:**

- What operations are needed?
- How many of these operations are going to be performed?

Want a Trie to perform operations

- **Dense: R-way Trie** vs **Sparse: DLB** (Won't see much repetition in **DENSE)**

Closed Addressing -> PERFECT HASHING!

Further Implementation Issues: Codeword Size

How long should codewords be?

1) **Use fewer bits:**
    a) Gives better compression earlier on!
    b) But, leaves fewer codewords available, which will hamper compression later on!
2) **Use more bits:**
    a) Delays actual compression until longer patterns are found due to large codeword size!
    b) More codewords available means that greater compression gains can be made later on in the process!

Variable Width Codewords:

1) Sounds eerily like vari**able-length codewords**
    a) Exactly what we set out to avoid!

2) Here we're talking about a different technique
(Length at a given instance is the same as **JUST ADJUSTING WIDTH**)

3) Example:
    a) Start out using 9 bit codewords
    b) When codeword 512 is inserted into the codebook, switch to outputting/grabbing 10 bit codewords!
    c) When codeword 1024 is inserted into the codebook, switch to outputting/grabbing 11 bit codewords!
    d) etc…

Even Further Implementation Issues: Codebook Size

1) What happens when we run out of codewords?
    a) Only 2^n possible codewords for n bit codes
    b) Even using variable width codewords, they can't grow arbitrarily large…
    - Call it off @ Some Point!
2) **Two Primary Options:**
    a) Stop adding new keywords, use the codebook as it stands
        ai) Maintains long already established patterns!
        aii) But if the file changes, it will **not be compressed as effectively**!
    b) Throw out the codebook and start over from single characters!
        bi) Allows new patterns to be compressed!
        bii) Until new patterns are built up, though, compression will be minimal!

The Showdown You've All Been Waiting For:

HUFFMAN vs. LZW

In general, LZW will give better compression.

1) Also better for compression archived directories of files!
    a) Why?
        ai) Very long patterns can be built up, leading to better compression!
        aii) Different files don't "**hurt**" each other as they did in Huffman!
            aiij) Remember our thoughts on using static tries?

So Lossless Compression Apps Use LZW?

1) Well, gifs can use it and pdfs!
2) Most dedicated compression applications use other algorithms:
    a) DEFLATE (combination LZ77 and Huffman)
        ai) Used by PKZIP and gzip!
    b) Burrows-Wheeler transforms
        bi) Used by bzip2
    c) LZMA
        ci) Used by 7-zip
    d) Brotli
        di) Introduced by Google
        dii) Based around a "… combination of a modern variant of the LZ77 algorithm, Huffman coding, and 2nd order context modeling"

DEFLATE etc. achieve even better general compression?

1) How much can they compress a file?
2) Better question: **How much can a file be compressed by any algorithm?**
3) No algorithm can compress every bitstream
    a) Assume we have such an algorithm
    b) We can use to compress its own output!
    c) And we could keep compressing its output until our compressed file is 0 bits!
        ci) Clearly this can't work!!

Can we reason about how much a file can be compressed?
YES! Using Shannon's Entropy!

Information Theory in a Single Slide…

1) Founded by **Claude Shannon** in his paper "A Mathematical Theory of Communication"
2) **Entropy** is a key measure in information theory!
    a) Slightly different from thermodynamic entropy!
    b) It's a **measure of the unpredictability of information content**!
    c) By losslessly compressing data, we represent the same information in less space!
    d) Hence, 8 bits of uncompressed text has less entropy than 8 bits of compressed data!

Entropy applied to a Language!

1) Translating a language into binary, the entropy is the average number of bits required to store a letter of the language!

2) Entropy of a message * length of message = **amount of information contained in that message**

3) On average, a lossless compression scheme cannot compress a message to have more than 1 bit of information per bit of compressed message!

4) Uncompressed, English has between 0.6 and 1.3 bits of entropy per character of the message!

A Final Note on Compression Evaluation:

1) Weissman Scores are MADE UP metric for Silicon Valley (TV)

String Pattern Matching:

General Idea:

1) Have a **pattern string p** of **length m**

2) Have a **text string t** of **length m**

3) Can we find an **index i** from **text string t** such that each of the m characters in the substring of **t** starting at **index i** matches each character in **pattern string p**?
- Can we find an **index** that corresponds to the beginning of the **pattern string**?

Example: Pattern Matching

Can we find the pattern "**fox**" in the text: "**the quick brown fox jumps over the lazy dog**"?
- Yes! We find that the pattern "**fox**" starts at **index 16** of the **text string!**

Simple Approach:

BRUTE FORCE

1) Start at the beginning of the **pattern** and the **text**!
2) Compare characters **left** to **right**!
3) If we get a **mismatch…**

Start again at the **2nd character of the text** and the **beginning of the pattern…**

IN OTHER WORDS…

1) Check each possible position in the text where the pattern could match, where it does in fact match!!

BRUTE FORCE CODE:

```
public static int bf_search(String pat, String txt) {
    int m = pat.length();
    int n = txt.length();
    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (txt.charAt(i + j) != pat.charAt(j))
                break;
        }
        if (j == m)
            return i; // found at offset i
    }
    return n; // not found
}
```

Analysis:

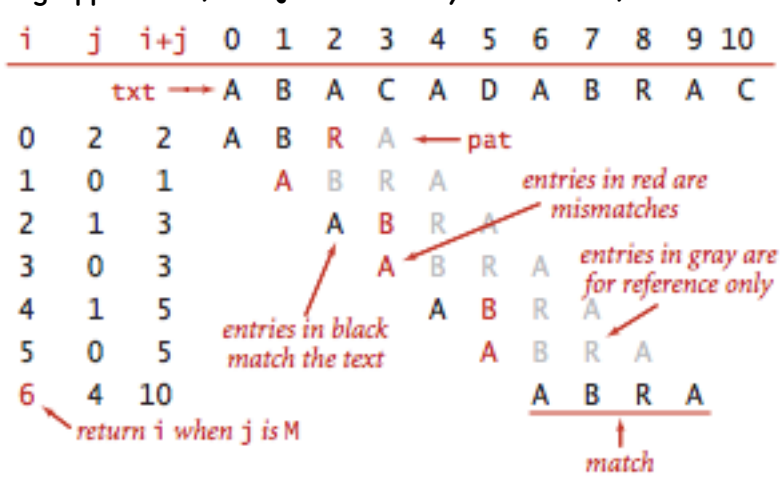
1) The program keeps one pointer i into the text and another pointer j into the pattern!

2) For each i, the program resets j to 0 and increments the pointer j corresponding to the pattern until either a **mismatch** is found, or we reach the end of the **pattern** ($j == m$)

3) If we reach the end of the **text string** ($i == n - m + 1$) before the end of the pattern, then there is **no match: The pattern does not occur in the text!**

- The convention is to return the **text string length** to indicate a **mismatch!**

In a typical text-processing application, the **j** index rarely increments, so the runtime is proportional to n.



Brute-force substring search

Brute Force Analysis:
Runtime? Depends on m and n
Worse Case: $\theta(n-m+1) = \theta(nm)$ **if n >> m**
Lots of repetition! Pattern is found at the last position in the **text string**, have to search through the entire pattern and text!

Average Case: $\theta(n+m) = \theta(n)$ **if n >> m**
Assume that mostly miss on the first pattern character!

Example: Worst Case
t = AAAAAAAAAB
p = AAAAB

i	j	i+j	0	1	2	3	4	5	6	7	8	9
		txt →	A	A	A	A	A	A	A	A	A	B
0	4	4	A	A	A	A	B ← pat					
1	4	5		A	A	A	A	B				
2	4	6			A	A	A	A	B			
3	4	7				A	A	A	A	B		
4	4	8					A	A	A	A	B	
5	5	10						A	A	A	A	B

Brute-force substring search (worst case)

Where do we Improve?
1) **Improve Worst Case!**
- Theoretically very interesting!!
- Practically doesn't come up often…

2) **Improve Average Case!**
    a) Much more practically helpful!
        ai) Especially, if we anticipate searching through large files!!

Improving the Worst Case: Knuth Morris Pratt (KMP)

Knuth-Morris-Pratt Algorithm (KMP): Substring Search

Goal: Want to avoid backing up the **text pointer** on a mismatch!

Main Idea:

In checking the pattern, we learn something about the characters in the text, take advantage of this knowledge to avoid backing up! (When we have a mismatch don't have to have pointer go through entire **text string**)

Insight: KMP Algorithm can decide ahead of time exactly how to restart the search, since that depends only on the pattern.

Backing Up the Pattern Pointer:

In KMP, the **text pointer** i is never backed up, and we use an array **dfa[][]** to record how far to back up the **pattern pointer** j when a mismatch is detected.

1) For every character c, $dfa[c][j]$ is the pattern position to compare against the next text position after comparing c with $pat.charAt(j)$.

2) During the search, $dfa[txt.charAt(i)][j]$ is the pattern position to compare with $txt.charAt(i+1)$ after comparing $txt.charAt(i)$ with $pat.charAt(j)$.

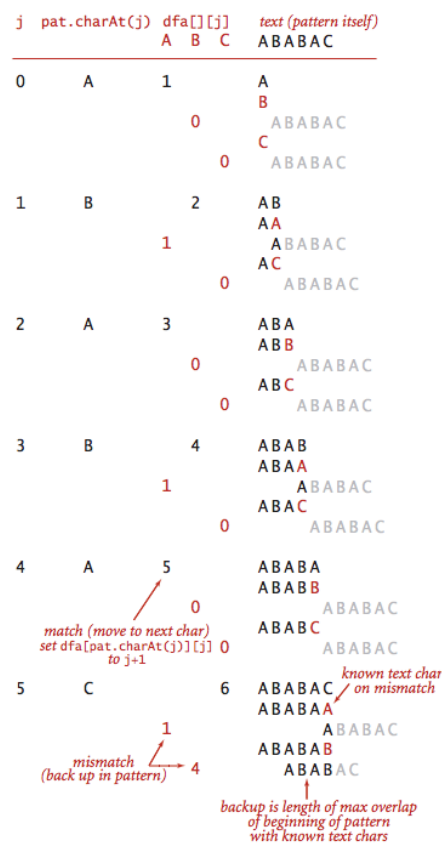
For a match, we want to just move onto the next character, so $dfa[pat.charAt(j)][j]$ is always $j+1$.

For a mismatch, we know $txt.charAt(i)$, but also the $j-1$ previous characters in the text: ***They are the first*** $j-1$ ***characters in the pattern starting at index 1.***

1) For each character c, imagine we slide a copy of the pattern over these j characters (those $j-1$ characters in the pattern followed by c we are deciding what to do when these characters are $txt.charAt(i-j+1 \ldots . i)$), from left to right, stopping when all overlapping characters match (or there are none).

- This gives us the next possible place the pattern could match.

The index of the pattern character to compare with $txt.charAt(i+1)$ ($dfa[txt.charAt(i)][j]$) is the number of overlapping characters.



Pattern backup for A B A B A C in KMP substring search

KMP Search Method:
Once the **dfa[][]** array is computed, we can now search…

When i and j point to mismatching characters (testing for a match beginning at position $\boldsymbol{i-j+1}$ in the **text string**), then the next possible position for a pattern match is beginning at position $\boldsymbol{i-dfa[txt.charAt(i)][j]}+1$.

By construction, the first $\boldsymbol{dfa[txt.charAt(i)][j]}$ characters at that position must match the first $\boldsymbol{dfa[txt.charAt(i)][j]}$ characters of the pattern, so there is no need to back up the $\boldsymbol{i}$ pointer: **We can simply set $\boldsymbol{j}$ to $\boldsymbol{dfa[txt.charAt(i)][j]}$ and increment $\boldsymbol{i}$, which is precisely what we can do when $\boldsymbol{i}$ and $\boldsymbol{j}$** point to matching characters!

How do we keep track of the text processed?
Build a Deterministic Finite-State Automata (DFA) storing information about the **pattern** searching information about the **pattern**
1) From a given state in searching through the pattern, if you encounter a mismatch, how many characters currently match the beginning of the pattern!

DFA Simulation:
Conveniently, our **dfa[][]** array precisely defines a DFA.
1) One state for each character in the pattern, each such state having one transition leaving it for each character in the alphabet.

2) For the substring matching DFAs under consideration, one of the transitions is a **match** transition (going from $\boldsymbol{j}$ **to** $\boldsymbol{j+1}$ and labeled with $\boldsymbol{pat.charAt(j)}$) and all others are **mismatch** transitions (**going left**)

KMP Code Analysis:

```
public int search(String txt)
{  // Simulate operation of DFA on txt.
   int i, j, N = txt.length();
   for (i = 0, j = 0; i < N && j < M; i++)
      j = dfa[txt.charAt(i)][j];
   if (j == M) return i - M;  // found
   else        return N;      // not found
}
```

1) The **states** correspond to character compares, **one** for each value of the pattern index.

2) The **transitions** correspond to changing the value of the pattern index.

3) When examining the **text character** i, when in the state labeled j, the machine does the following: "**Take the transition to** $dfa[txt.charAt(i)][j]$ **and move to the next character (by incrementing** i**).**"

4) For a **match transition**, we move to the right one position, b/c $dfa[pat.charAt(j)][i]$ is **always** $j+1$.

5) For a mismatch transition, we move to the left.

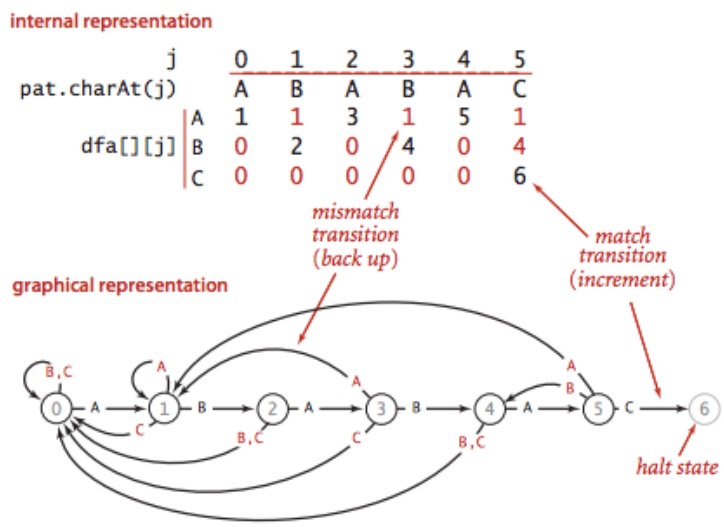
6) The DFA reads the **text characters** one at a time, from **left to right**, moving to a new state each time it reads a character.

7) We also include a **HALT** state m that has **no transitions**.

8) DFA starts @ State 0, if it reaches State "**m**": Pattern is found, DFA recognizes the pattern!

9) If DFA reaches end of text before reaching State "**m**", then pattern does not appear as a **substring of the text**.'

10) Each pattern corresponds to DFA (represented by $dfa[\,][\,]$ array that gives transitions)



DFA corresponding to the string A B A B A C

Representing the DFA in Code:

Represented as a 2D array:

- $dfa[curr_text_char][pattern_counter] = new_pattern_counter$

Storage Needed: mR (Rows * Cols)

Runtime:

$\theta(n)$ **worst case (Linear proportional to the length of the text string)**

DFA: At least **mR**

Mismatched Character Heuristic:

How well pattern matching works depends on the pattern and text at hand…

1) What do we do in the general case after a mismatch?

    a) Consider:

    t = "XYXYXYZXXXXXXXXXXXXXX"

    p = "XYXYZ"

    b) If the mismatched **does appear in** p, need to "slide" to the right to the next occurrence of that character in p!

        bi) Requires us to **pre-process the pattern**! (Rightmost occurrence!)

            bij) Create a **right array**!

For all the characters **not in the pattern:**

```
for (int i = 0; i < R; i++)
    right[i] = -1;
```

For all the characters that **are in the pattern:**

```
for (int j = 0; j < m; j++)
    right[p.charAt(j)] = j;
```

Another Approach: Boyer Moore

What if we compare starting at the end of the pattern?

For Example:

t = "ABCDEVABCDWABCXABCDYABCDZ"

p = "ABCDE"

1) **V** does not match **E**

    a) Furthermore, **V** is nowhere in the pattern…

    b) So, skip ahead **m** positions with 1 comparison!

        bi) **Runtime?**

            bij) **Best Case:** n/m

When searching through text with a large alphabet, we will often come across characters not in the pattern.

1) One of Boyer Moore's heuristics takes advantage of this fact!

    a) Mismatched character heuristic!

    - Use information when we get a mismatch to determine how much to skip ahead!

Example: Missed Character Heuristic

Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
Pattern: A B C D E
Rightmost Array: [0, 1, 2, 3, 4, -1, -1, ...]

Let's try using the **Missed Character Heuristic!**

Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
A B C D E
Oh no, we have a mismatch! Now we have to figure out how much to offset!

Well, E is 4 in the Rightmost Array, and we don't have X in our pattern, so it's -1...
So, our offset is 4 - - 1 = 5...

Now let's try matching again!!
Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
A B C D E
A B C D E
Oh no! We have another mismatch!! Now we have to figure out how much to offset!

Well, E is 4 in the Rightmost Array, and we have C in our pattern, so it's 2...
So, our offset is 4 - 2 = 2...

Now let's try matching again!!
Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
A B C D E
A B C D E
A B C D E
Oh no! We have another mismatch!! Now we have to figure out how much to offset!

Well, E is 4 in the Rightmost Array, and we have B in our pattern, so it's 1...
So, our offset is 4 - 1 = 3...
Now let's try matching again!!

```
Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
      A B C D E
                A B C D E
                    A B C D E
                          A B C D E
```

Oh no! We have another mismatch!! Now we have to figure out how much to offset!

Well, E is 4 in the Rightmost Array, and we don't have Y in our pattern, so it's -1…
So, our offset is 4 -- 1 = 5…

Now let's try matching again!!

```
Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
      A B C D E
                A B C D E
                    A B C D E
                          A B C D E
                                    A B C D E
```

Oh no! We have another mismatch!! Now we have to figure out how much to offset!

Well, B is 2 in the Rightmost Array, and we have E in our pattern, so it's 4…
So, our offset is 2 - 4 = -2… But wait!! This is a negative index!! So, let's go forward by 1…

Now let's try matching again!!

```
Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
      A B C D E
                A B C D E
                    A B C D E
                          A B C D E
                                    A B C D E
                                      A B C D E
```

Oh no! We have another mismatch!! Now we have to figure out how much to offset!

Well, E is 4 in the Rightmost Array, and we have A in our pattern, so it's 0…
So, our offset is 4 - 0 = 4…

Now let's try matching again!!

```
Text: A B C D X A B C D C A B C D Y A E C D E A B C D E
      A B C D E
                A B C D E
                  A B C D E
                      A B C D E
                                A B C D E
                                  A B C D E
                                        A B C D E
```

YAY! We don't have any mismatches, so found the substring!!

Runtime for Missed Character:

1) What does the worst case look like?
   a) We have to compare all of the characters in the pattern!
2) **Runtime:** $\theta(mn)$
   a) Same as brute force!
3) This is why missed character is **only one** of **Boyer Moore's** heuristics!
   a) **Galil Rule**: Hybrid of KMP and Boyer-Moore!
4) **See BoyerMoore.java**

BoyerMoore.java Analysis:

```
public int search(String txt)
{  // Search for pattern in txt.
   int N = txt.length();
   int M = pat.length();
   int skip;
   for (int i = 0; i <= N-M; i += skip)
   {  // Does the pattern match the text at position i ?
      skip = 0;
      for (int j = M-1; j >= 0; j--)
         if (pat.charAt(j) != txt.charAt(i+j))
         {
            skip = j - right[txt.charAt(i+j)];
            if (skip < 1) skip = 1;
            break;
         }
      if (skip == 0) return i;          // found.
   }
   return N;                            // not found.
}
```

Starting Point:
To implement the **mismatched character heuristic**, we use an **array right[ ]** that gives, for each character in the alphabet, the index of its **rightmost occurrence** in the pattern (or -1 if the character is not in the pattern)!

This value tells us precisely how far to skip if that character appears in the text and causes a mismatch during the string search!

To initialize the **right [ ]** array, we set all entries to -1, and then, for j **from** 0 to M-1, set $right[pat.charAt(j))]$ to j

Substring Search:
With the **right[ ] array precomputed/preprocessed,** the implementation of Boyer-Moore is straightforward…
1) Have an index i moving from left to right through the text
2) Have an index j moving from left to right through the pattern
3) Inner loop tests whether a pattern aligns with the text at position i.

4) If $txt.charAt(i+j)$ is equal to $pat.charAt(j)$ for all j from $M-1$ down to 0, then there is a match!

5) Otherwise, there is a character mismatch, and we have one of the following **three cases**:

a) If the character causing the mismatch is not found in the pattern, we can slide the pattern $j+1$ positions to the right (increment i by $j+1$)

ai) Anything less would align that character with some pattern character

aii) This move aligns some known character at the beginning of the pattern with known characters at the end of the pattern (further increases i)!

b) If the character c causing the mismatch is found in the pattern, we use the **right[ ]** array to line up the pattern with the text so that character will match its rightmost occurrence in the pattern!

bi) This is done by incrementing i by $j - right[c]$

c) If the computation would not increase i, we just increment i instead, to make sure that the pattern always slides at least one position to the right!

Rabin Karp: Fingerprint Search

Hash-based substring search!

1) Compute a hash function for the pattern and look for a match by using the same function for each possible **M**-character substring of the text.

2) If we find a text substring with the **same hash value** as the patter, we can check for a match.

Process is equivalent to storing the pattern in a Hash Table, then doing a search for each substring of the text, but we do not need to reserve the memory for the has table, b/c it would only have one entry.

Basic Plan:

A string of length **M** corresponds to an **M-digit** base-**R** number.

1) To use a hash table of size **Q** for keys of this type, need a hash function to convert an **M-digit** base-**R** number to an int value between 0 and **Q - 1**.

Modular Hashing (via Horner's Method) provides an answer:

1) Take the remainder when dividing the number by Q

- In practice, we use a random prime **Q**, taking as large a value as possible while avoiding overflow.

Example: Rabin-Karp

Say we want to search for 26535, with a table size of 997.

	pat.charAt(j)					
j	0	1	2	3	4	
	2	6	5	3	5	% 997 = 613

If the text string's hash value ≠ pattern's hash value, shift over one value, and compare hash values…

	txt.charAt(i)															
i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
0	3	1	4	1	5	% 997 = 508										
1		1	4	1	5	9	% 997 = 201									
2			4	1	5	9	2	% 997 = 715								
3				1	5	9	2	6	% 997 = 971							
4					5	9	2	6	5	% 997 = 442						
5						9	2	6	5	3	% 997 = 929					
6 ← *return* i = 6							2	6	5	3	5	% 997 = 613 *match*				

Basis for Rabin-Karp substring search

Well, that was simple…
Is it efficient?
1) Nope! Practically worse than brute force!
    a) Instead of **mn** character comparisons, we perform **n** hashes of **m** character strings!

Computing the Hash Function:

```
private long hash(String key, int M)
{  // Compute hash for key[0..M-1].
   long h = 0;
   for (int j = 0; j < M; j++)
      h = (R * h + key.charAt(j)) % Q;
   return h;
}
```

Horner's method, applied to modular hashing

1) Hash function for an **M-digit** base-**R** number represented as a **char array** in time proportional to **M** is computed.
- **M** is passed as an argument so that method can be used for both **pattern** and the **text**!

2) For each digit in the number, we multiply by **R,** add the digit, and take the modulus Q.

3) Don't have to recompute hash of each substring!

Example: Modular Hashing
abcde only hash 4 at a time!
- Hashed becomes 'a' * 'R'^3 + 'b'*R^2 + 'c'^R + 'd'
Want to hash next value…
- Hashed is ('a' * 'R'^3 + 'b'*R^2 + 'c'^R + 'd')* R – a*R^4 + e

OR Hashed becomes ('a' * 'R'^3 + 'b'*R^2 + 'c'^R + 'd') mod Q
Want to hash next value…
- Hashed is (('a' * 'R'^3 + 'b'*R^2 + 'c'^R + 'd')* R) mod Q + e

Implementation of Rabin-Karp:

```
private int search(String txt)
{  // Search for hash match in text.
   int N = txt.length();
   long txtHash = hash(txt, M);
   if (patHash == txtHash) return 0;            // Match at beginning.
   for (int i = M; i < N; i++)
   {  // Remove leading digit, add trailing digit, check for match.
      txtHash = (txtHash + Q - RM*txt.charAt(i-M) % Q) % Q;
      txtHash = (txtHash*R + txt.charAt(i)) % Q;
      if (patHash == txtHash)
        if (check(i - M + 1)) return i - M + 1; // match
   }
   return N;                                    // no match found
 }
```

1) **Constructor** computes a hash value $patHash$ for the pattern; it also computes the value of $R^{M-1} mod\ Q$ in the variable RM.

2) The **hashSearch** method begins by computing the hash function for the first **M** characters of the text and comparing the value against the hash value for the pattern.

3) If that is not a match, proceeds through the text string!

What about Collisions?

Note, we're not storing any values in a Hash Table…

1) So, increasing Q doesn't affect memory utilization (No harm in picking in large Q)

- Make Q really big and change of collision becomes really small!

(But not zero!)

2) So, do a char-by-char comparison on a hash match just to be sure!

Worst Case: $\theta(nm)$

- Back to **brute force esque runtime…**

Assorted Casinos:

Two options:

1) **Las Vegas: Guaranteed correct, probably fast** (Not sure? Make sure!!)

- Do a char-by-char comparison after hash match!

2) **Monte Carlo: Guaranteed fast, probably correct!** (Cross your fingers!)

Boyer Moore vs Rabin Karp

1) BOYER MOORE!!

Priority Queues:

Mentioned PQs in Building Huffman Tries:

Primary Operations needed:

1) **Insert**

2) **Find Item with Highest Priority (FETCH)**

- E.g. **findMin() (Min PQ -> smallest at root)** or **findMax() (Max PQ -> largest at root)**

3) Remove an Item with the **highest priority**

- E.g. **removeMin()** or **removeMax()**

How do we implement these operations?

SIMPLEST APPROACH: ARRAYS ;)

Data Structure:	Insert	Find	Remove	Usage in Huffman Tree Generation
Unsorted Array PQ	**Amortized** $\theta(1)$ Add new item to end of array!	$\theta(n)$ Search for highest priority item! (Min or Max)	$\theta(1)$ Search for highest priority item and delete!	$\theta(n^2)$ Boils down to **n** insertions and 2**n** removals!
Sorted Array PQ	$\theta(n)$ Add new item in appropriate sorted order! - Shift elements to open space for new element!	$\theta(1)$ Return the item at the end of the array! - Highest priority at the end of the array (max PQ) -> Descending for (min PQ)	$\theta(1)$ Return and delete the item at the end of the array!	$\theta(n^2)$ Insert **n** trees, adding more trees causes n^2. - Need to maintain relative order (even if it's sorted!)
Heap PQ	$\theta(log(n))$: At most traverse entire height of tree!	$\theta(1)$: Simply the root of the tree!	$\theta(log(n))$: At most traverse entire height of tree!	**Ummmm…**

What other options are there?

1) **What about BST?**

   a) **Insert:** $\theta(log(n))$ **Average, Worst Case** $\theta(n)$ if unbalanced!

   b) **Find:** $\theta(log(n))$ **Average, Worst Case** $\theta(n)$ **if unbalanced!**

   c) **Remove:** $\theta(log(n))$ **Average, Worst Case** $\theta(n)$ **if unbalanced!**

2) So, average case, all operations are $\theta(log(n))$…

- No constant time operations :(

- Worst Case is $\theta(n)$ -> Not self-balancing!

This is asymptotically fine, but it's a real hassle to rebalance the tree

Is a BST Overkill?

1) Find and Remove only need the **HIGHEST PRIORITY ITEM,** not find/remove **any** item!

- Rearrange to favor the highest priority!
    a) Can we take advantage of this to improve our runtime?
    YES!!

The Heap:

A **heap** is a **complete binary tree** such that for each node **T** in the tree:

- A **complete binary tree** is a full tree, where the **leaf nodes** are as far left as possible!
- **Full until the 2nd last level!** (Last level is allowed to be partially full!)

1) **T.item** is of a higher priority than **T.right_child.item**!
2) **T.item** is of a higher priority of **T.left_child.item!**

"Transitivity says T.item is higher priority than all of its descendants"

It does not matter how T.left_child.item relates to T.right_child.item!

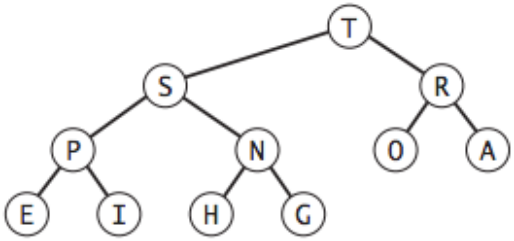
1) This is a relaxation of the approach needed by a **BST (where it's sorted)!**

The HEAP Property: Heap is PARTIALLY SORTED!

"Every node should be higher priority than its children!"

Example: Heap Representation

This is an example of a Heap…



A heap-ordered complete binary tree

Heap PQ Runtime:
1) **Find is Easy!!!**
- It's just the **root of the tree!**
So, it's just: $\theta(1)$!

2) **Remove** and **Insert** are not quite so trivial!
- The tree is modified and the **heap property** must be maintained!

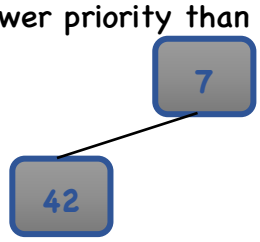
"We have to process add and remove operations to maintain Tree Shape, then fix the Heap Property!" (Isn't this kind of like with B-trees?)

Heap Insertion:
1) Add a new node at the next available leaf!
- Maintain complete shape!
2) Push the new node up the tree until it supports the **heap property!**
- **Float Up (Swim)** until the **heap property is maintained!**

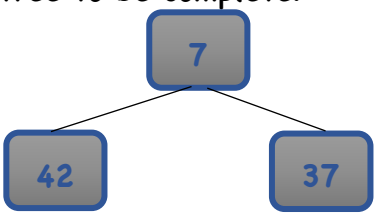
Min Heap Insertion:
Say we want to insert the following…
7, 42, 37, 5, 8, 15, 12, 9, 3

1) We want to insert **7…**
Our heap is currently empty, so we can just insert 7 as the root!

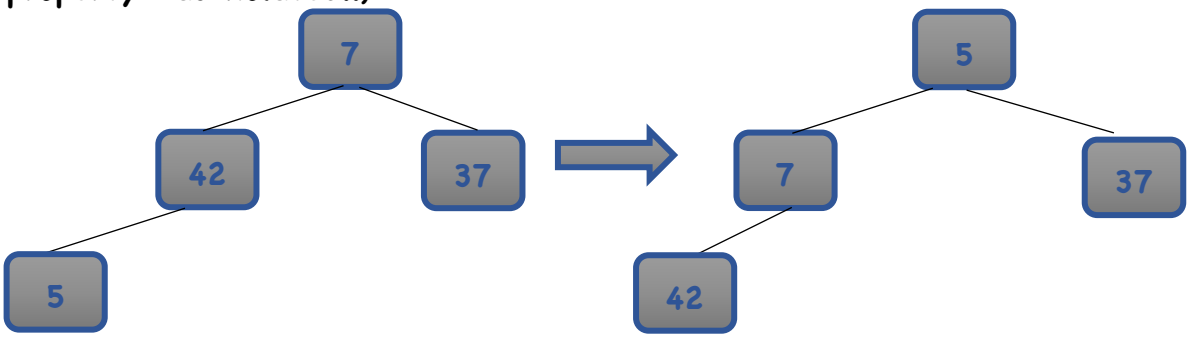
2) Now we want to insert 42…
42 is a lower priority than 7, so we insert it to the left…



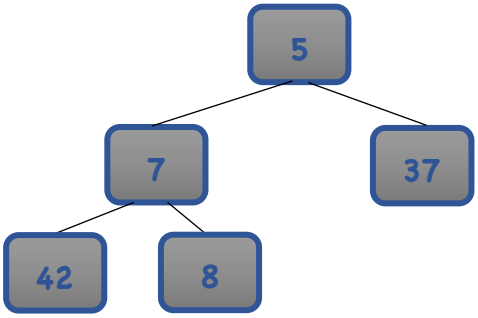
3) Now we want to insert 37…
Since 37 has a lower priority than 7, we're gonna put it on the right side, since we want the tree to be complete!



4) Now we want to insert 5…
We're gonna initially put 5 as a left child to 42… But then, since 5 is lower priority than 7, we swim 5 up, swapping 42 and 5, and then 7 with 5… (b/c the **heap property was violated!!**)

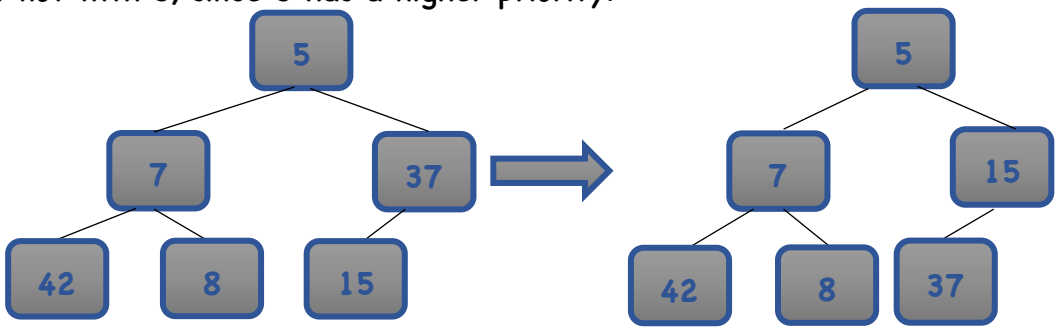


5) Now we want to insert 8…
Since 8 has a lower priority than 5, and the left side of the tree is not complete yet, we insert 8 as a right child to 7…



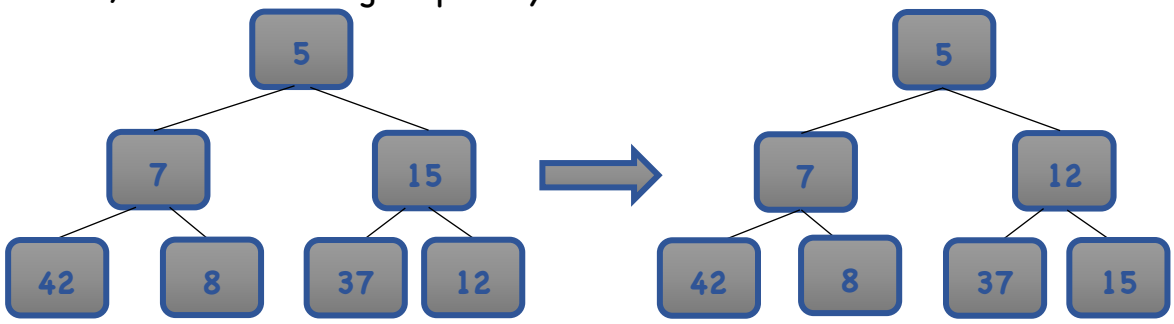
6) Now we want to insert 15…

Since 15 has a lower priority than 5, and we see that the left subtree is complete, we now insert into the right subtree. So, 15 is initially put as a left child to 37. But wait! 15 has higher priority than 37, so we need to **swim** 15 up, swapping with 37, but not with 5, since 5 has a higher priority!

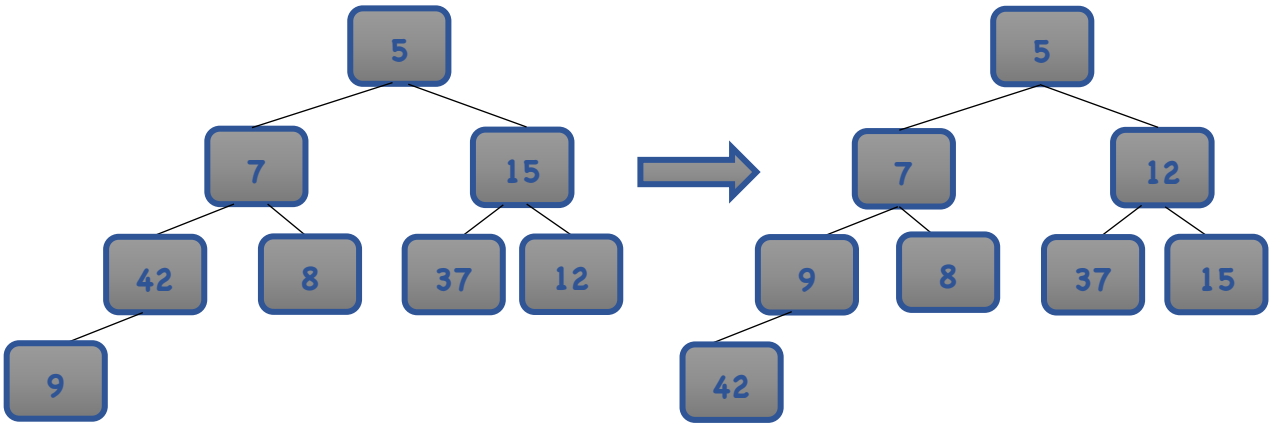


7) Now we want to insert 12…

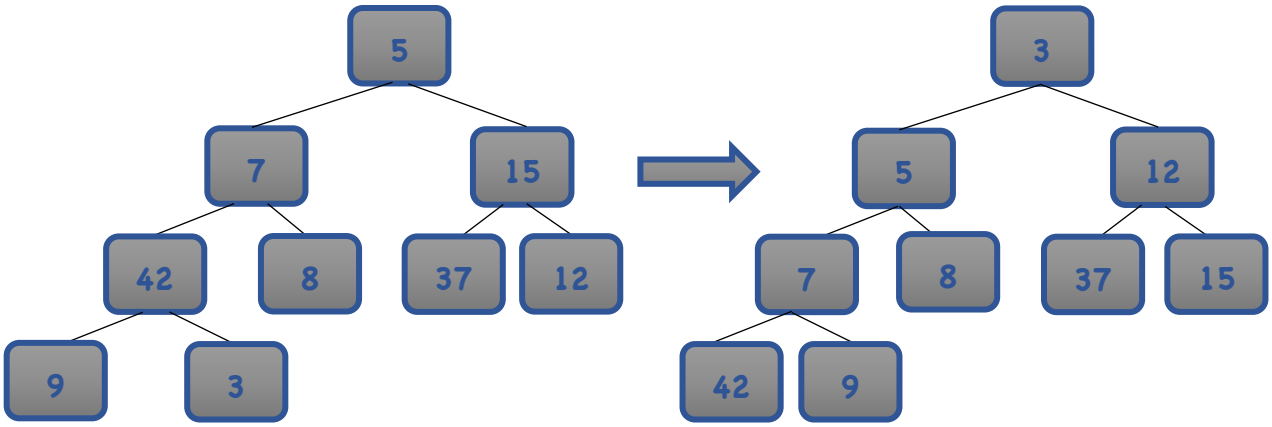
Since 12 has a lower priority than 5, and we see that the left subtree is complete, we now insert into the right subtree. So, 12 is put as a right child to 35. But wait! 12 has higher priority than 15, so we need to **swim** 12 up, swapping with 15, but not with 5, since 5 has a higher priority!



8) Now we want to insert 9…
Since 9 has a lower priority than 5, and we see that the left subtree and right subtrees are complete, we now insert into the left subtree. So, 9 is put as a left child to 42. But wait! 9 has higher priority than 42, so we need to **swim** 9 up, swapping with 42, but not with 7, since 7 has a higher priority!



9) Finally, we want to insert 3…
Since 3 has a higher priority than 5, and we see that the left subtree is not complete, we now insert into the left subtree. So, 3 is put as a right child to 9. But wait! 3 has lower priority than 42, so we need to **swim** 9 up, swapping with 42, but wait again! 3 also has a higher priority than 7, so we swap with 7! Oh no, not again! 3 also has a higher priority than 5, so we swap 5 with 3!



Heap Removal:

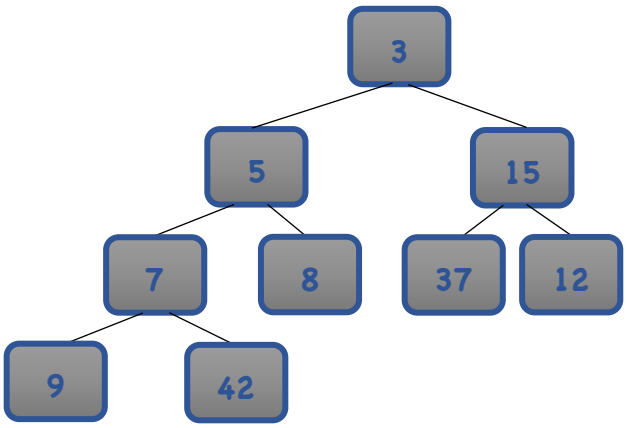
It's pretty tricky to delete the root!

1) Let's simply overwrite the root with the item from the last leaf **on level** and then delete the last leaf!

- But then the root is violating is violating the **heap property!**
    a) So, we **push the root down the tree** until it supports the **heap property!**
    - Sink down the tree until it satisfies the **Heap Property!**

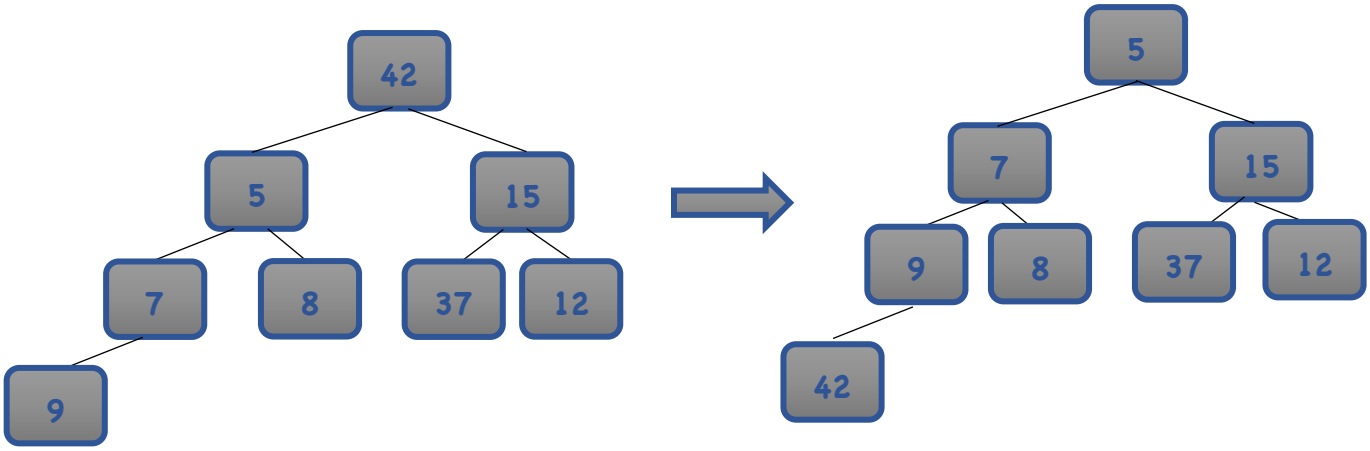
Minimum Heap Removal:

Suppose we have the following **heap**:



Say we want to delete the root 3…

1) Swap 3 with 42…
2) Delete 3 from the heap…
3) Now the **heap property is violated so** we just have to adjust!!



Heap PQ Runtimes:

1) **Find:** $\theta(1)$

2) **Insert and Remove:**

a) The heiht of a **Complete Binary Tree** is **log(n)**

b) At most, upheap and downheap operations traverse the height of the tree!

c) **Hence, insert and remove are** $\theta(log(n))$

Heap Implementation:

1) Simply implement tree nodes like for **BST**

- Requires overhead for dynamic node allocation
- Also, must follow chains of parent/child relations to traverse the tree

2) Heap will be a **Complete Binary Tree**

- Can easily represent a complete binary tree using an array!

In Level Order: L -> R, then T -> D!

Storing a Heap in an Array:

1) Let's number all of the nodes row-wise starting at zero!

2) Now let's use these numbers as indices in the array!

3) Now, for node at index i:

a) $parent(i) = \lfloor (i-1)/2 \rfloor$: Round down if the result is not an integer!

b) $left_child(i) = 2i + 1$: **ODD ENTRIES**

c) $right_child(i) = 2i + 2$: **EVEN ENTRIES**

- For arrays indexed from zero!

ROOT	ROOT's LCHILD	ROOT's RCHILD	LCHILD's LCHILD	LCHILD'S RCHILD	RCHILD's LCHILD	RCHILD's RCHILD	LCHILD'S LCHILD	LCHILD's RCHILD

Heap Sort:

1) **Heapify the Numbers**

- **MAX heap** to sort **ascending**
- **MIN heap** to sort **descending**

2) **Remove** the **root**

- **Don't actually delete the leaf node**: "Swap, and pretend the heap ends at that element"
- **Readjust for the Heap Property**

3) **Consider the heap to be from 0… to length - 1**

4) **REPEAT**

Heap Sort Analysis:

Runtime:

1) **Worst Case:** $\theta(n\,log(n))$

In Place?

- Yes, no extra array!

Stable?

- Nope, doesn't maintain relative order!

Like QuickSort, but $\theta(n\,log(n))$

- Lower multiplicative term with QuickSort though…

Example: Indirection Setup

Suppose I'm shopping for a new video card and want to build a heap to help me keep track of the lowest price available from different stores!

1) Let's say I keep objects of the following type in the heap:

```
class CardPrice implements Comparable<CardPrice>{
   public String store;
   public double price;
   public CardPrice(String s, double p) { … }
   public int compareTo(CardPrice o) {
      if (price < o.price) { return -1; }
      else if (price > o.price) { return 1; }
      else { return 0; }
   }
}
```

Storing Objects in a PQ:

What if we wanted to **Update an Object? (The Heap Property might be violated!)**

1) What is the runtime to find an arbitrary item in a heap?

$\theta(n)$: **It's Sequential Search!**

Hence, updating an item in the heap is **AT LEAST** $\theta(n)$

2) **Can we improve this?**

- Yep! If we back up the PQ with something besides a **heap!**
- Or develop some clever workaround!

Indirection:

1) Maintain a second data structure that **maps Item IDs** to each item's **current position in the heap**!

2) **Creates an INDEXABLE PQ.**

Indirection Example:

n = new CardPrice("NE", 333.98);

a = new CardPrice("AMZN", 339.99);

x = new CardPrice("NCIX", 338.00);

b = new CardPrice("BB", 349.99);

Indirection Table currently looks like:

Indirection
"NE": 0
"AMZN": 2
"NCIX": 1
"BB": 3

PQ Array looks like:

b	a	n	x					

Update Price for NE: 340…

Indirection Table currently looks like:

Indirection
"NE": 2
"AMZN": 1
"NCIX": 0
"BB": 3

Update Price for NCIX: 345…

Indirection Table currently looks like:

Indirection
"NE": 1
"AMZN": 0
"NCIX": 2
"BB": 3

Update Price for BB: 200…

Indirection Table currently looks like:

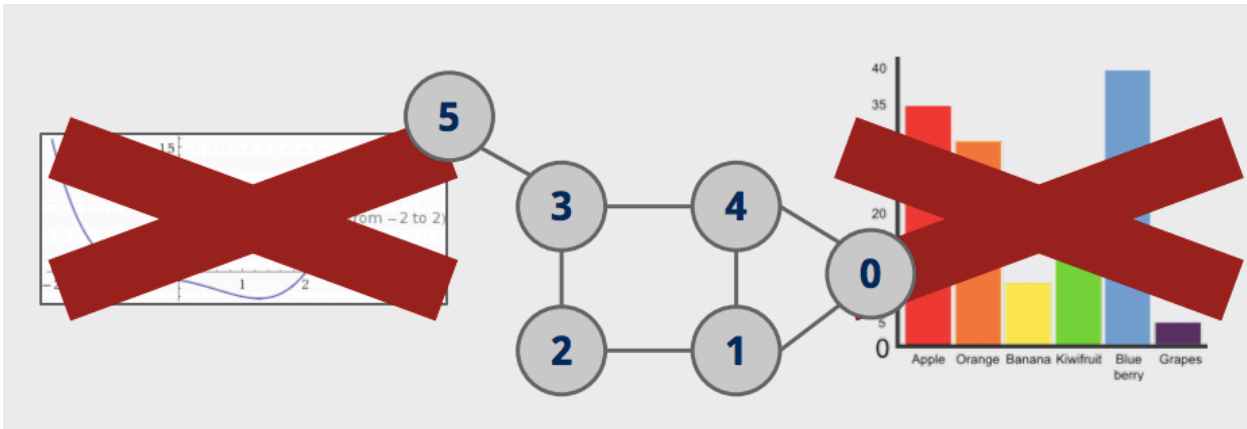
Indirection
"NE": 2
"AMZN": 1
"NCIX": 3
"BB": 0

Runtime:

Add & Remove: $\theta(log(n))$

Find & Update: $\theta(log(n))$

Graphs:



Graphs are a generalization of trees!

Graphs:

1) A **graph** $G = (V, E)$

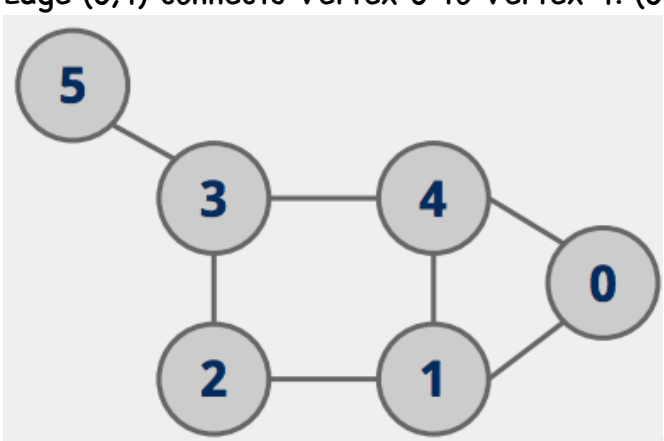
- Where **V** is a set of vertices!
- **E** is a set of **edges** connecting **vertex pairs**

2) **Example:**

$V = \{0, 1, 2, 3, 4, 5\}$

$E = \{(0, 1), (0, 4), (1, 2), (1, 4), (3, 4), (3, 5)\}$

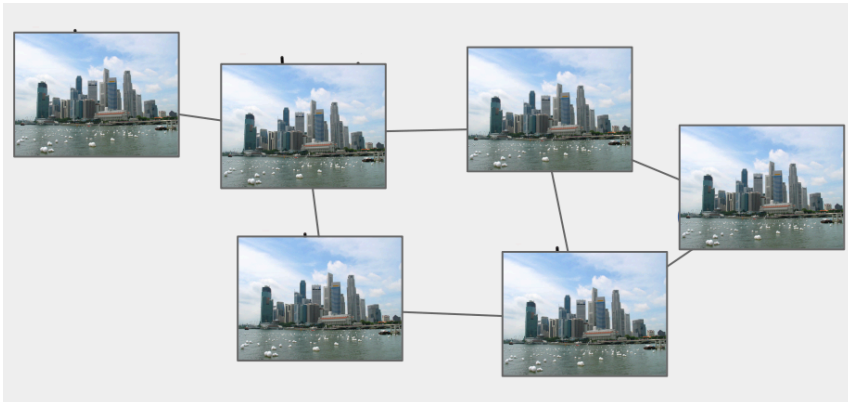
Edge (0,4) connects Vertex 0 to Vertex 4! (0,4) is like a small path!



But Why???

Graphs can be used to model many different scenarios!

- **BIG APPLICATION: Social Networks!**



Some Definitions:

1) **Undirected Graph:**

- **Transitivity**: Edges are unordered pairs: $(A, B) == (B, A)$

2) **Directed Graph:**

- **Antisymmetric & AntiTransitive:** Edges are ordered pairs: $(A, B) \neq (B, A)$

3) Adjacent vertices, or neighbors

- Vertices that are connected by an edge!

Graph Sizes:

Let $v = |V|$ and $e = |E|$

1) Given v, what are the minimum/maximum sizes of e?

a) Minimum value of e?

- **Well,** there's nothing saying there can't be NO EDGES
- **So, zero!**

b) **Maximum of** e?

bi) Well, it depends…

bij) Are **self-edges allowed**?

bijj) Is it a **Directed Graph** or an **Undirected Graph?**

If **Directed:** V^2

Undirected: vC_2

Assume that **directed graphs** **may have self-edges (come back to itself, you can follow yourself)**, while **undirected graphs** **do not (can't friend yourself)**

More Definitions:

1) A **graph** is considered **sparse** if:

- $e \leq vlog(v)$: **Vague Connectivity, Closer to No edges than ALL EDGES.**

(More about the trend than the actual number!!)

2) A **graph** is considered **dense** as it approaches the **maximum number of edges**!

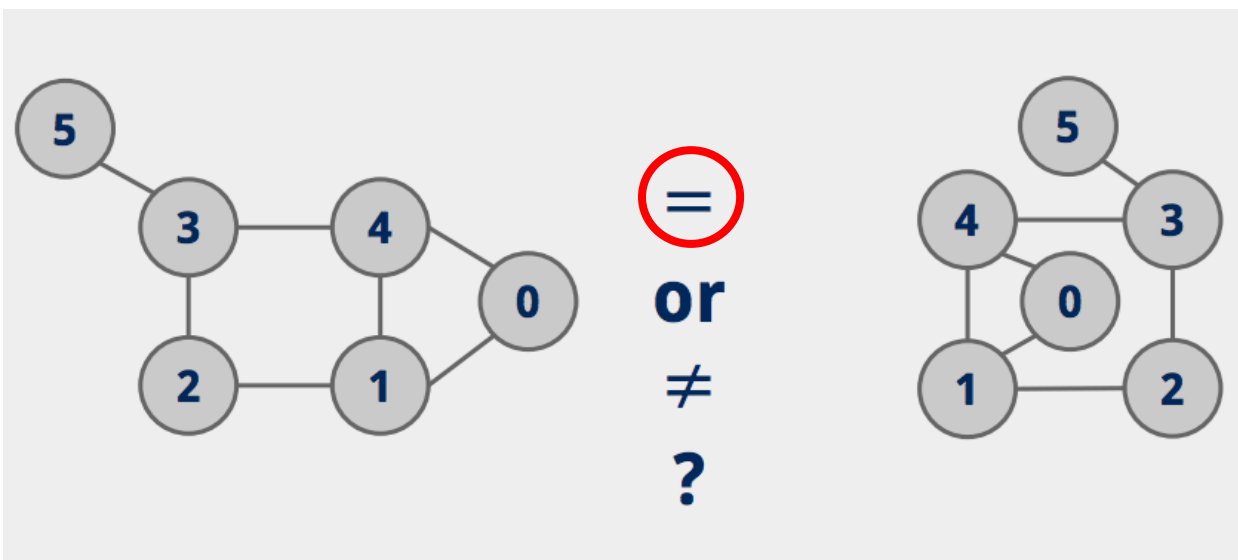
- $e == MAX(edges) - \varepsilon$, **where** ε is a small constant!

3) A **complete graph** has the maximum number of edges!

Not being dense DOES NOT IMPLY sparse!

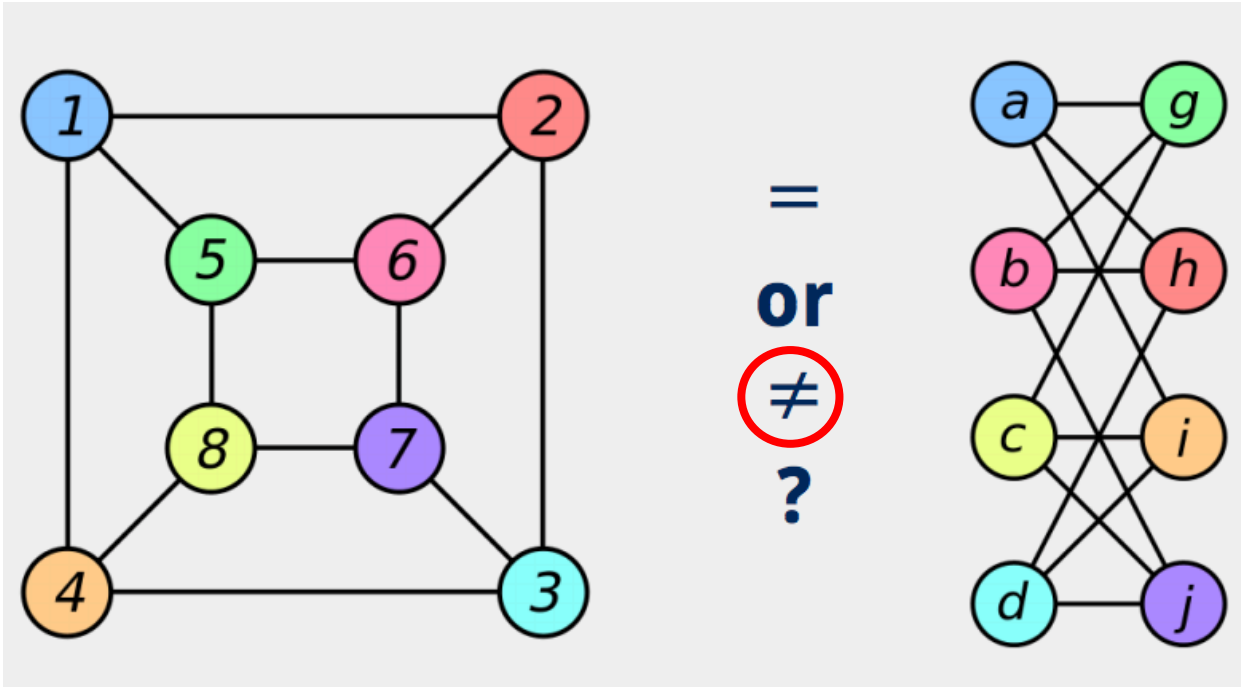
- Complete does imply Dense...

Are the Two Graphs Equal?



YEP!! Arrangement doesn't matter!!

Are the Two Graphs Related?



Nope, must be Isomorphic!
1) Exhibit **equivalence relation & bijection**
2) "You should explore the **Whitney Graph Isomorphism Theorem**" ~Garrison

Representing Graphs:
1) Trivially, graphs can be represented as:
- A **List of Vertices**!
- A **List of Edges**!

2) Performance?
- Assume we're going to be analyzing **static graphs**!
(No inserts and removes)
- So, what operations should we consider?
Is there an edge between 2 vertices, A & B**?** $\theta(E)$

Is there a path? $\theta(E)$

Is there a neighbor? $\theta(E)$

Even More Definitions:

1) **Path:** A sequence of **adjacent vertices**!

2) **Simple Path:** A path in which **no vertices are repeated**!

3) **Simple Cycle:** A **simple path** with the **same first and last vertex**!

4) **Connected Graph:** A graph in which a path exists between **all vertex pairs!**

5) **Connected Component: Connected subgraph of a graph!**

6) **Acyclic Graph:** A graph with **no cycles!**

7) **Tree:** A connected, acylic graph! (Often "rooted")
- Has exactly $v - 1$ edges (Every non-root has one parent)

Graph Traversal:
What's the best order to traverse a graph?
Two Primary Approaches:
1) **Depth-First Search (DFS):**
- **"Dive" as deep as possible into the graph first**
- **Branch when necessary!**

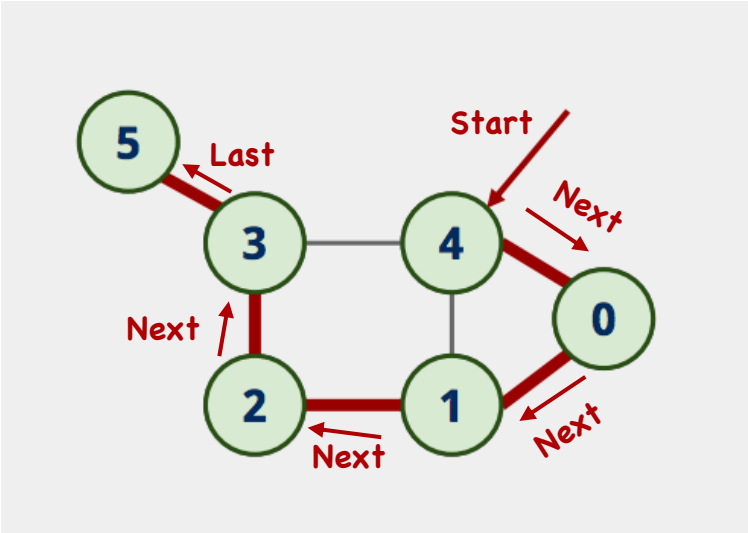
2) **Breadth-First Search (BFS):**
- **Search all directions evenly**
(From i, visit all of $i's$ neighbors, then all of their neighbors, etc…)
- I.e., consider all options!

DFS:
1) We've already seen and used DFS a lot (for trees)!
- For tries…
- For Huffman encoding…

2) Can easily be implemented **recursively**
- For each node, visit first unseen neighbor!
- Backtrack at **dead ends!** (I.e., nodes with **no unseen neighbors**)
    - Try the next unseen neighbor **after backtracking!**

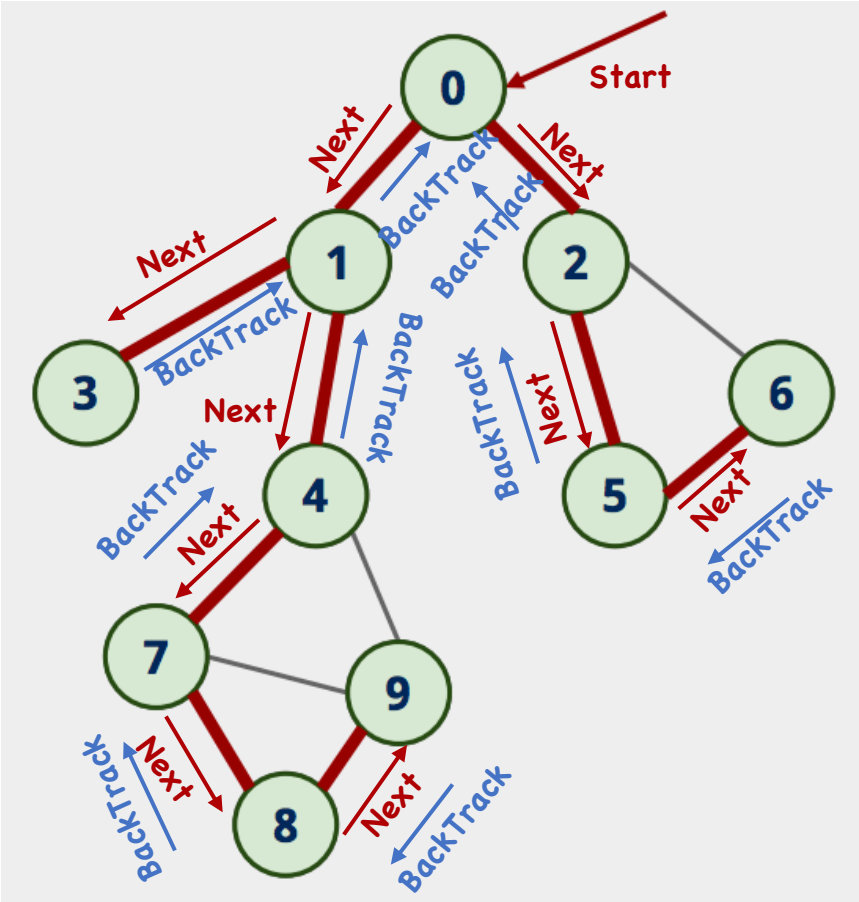
Example: DFS

Let's start from vertex 4, and suppose we want to traverse based on ascending order (numerically)



If we've already visited the vertex, ignore it!!

Example: DFS 2



DFS

- Subgraph is a Tree!
- Specifically, a **Spanning Tree!**

BFS:

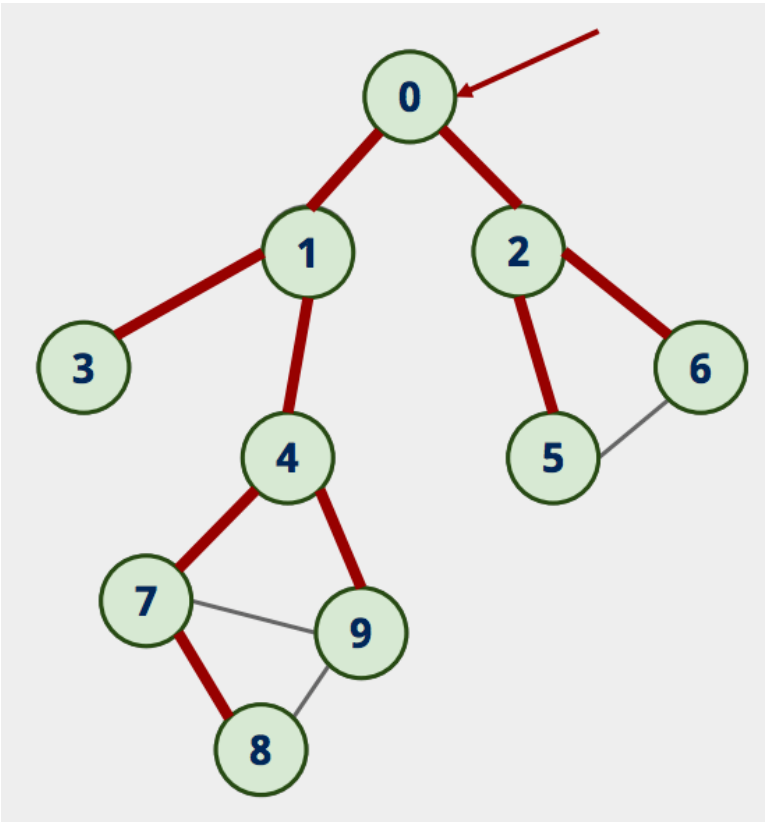
Can easily be implemented using a **queue**!

1) For each node visited, add all of its neighbors to the **queue**

- Vertices that have been seen but not yet visited are said to be **on the fringe**.

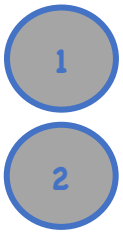
2) Pop head of the queue to be the next visited vertex!

Example: BFS



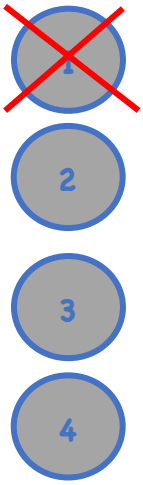
1) Visit 0, **Enqueue 0's neighbors**, and dequeue 0...

Q



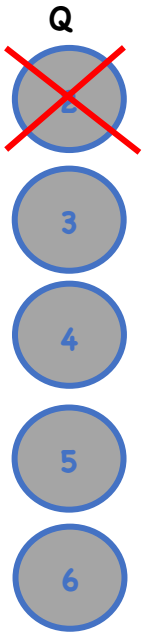
2) Visit 1, and Enqueue all of 1's neighbors, dequeue 1

Q

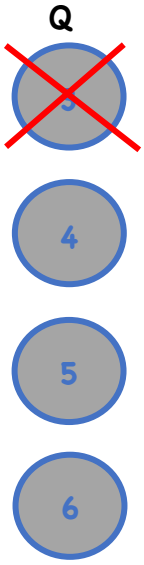


A BFS actually generates the Shortest Path from its **Root to Any Vertex!**

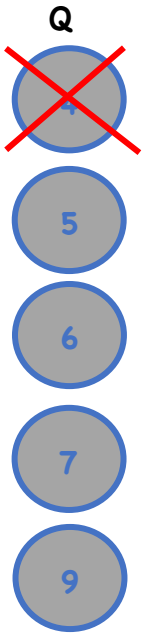
3) Visit 2, and enqueue all of 2's neighbors, dequeue 2..



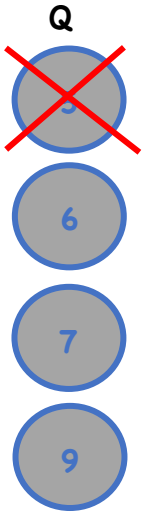
4) Visit 3, and attempt to enqueue all of 3's neighbors, since 0 has already been visited, simply dequeue 3…



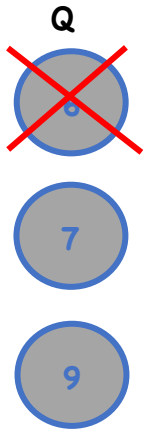
5) Visit 4, and enqueue all of 4's neighbors, dequeue 4…



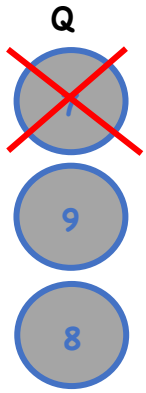
6) Visit 5, and attempt to enqueue all of 5's neighbors, since 6 and 2 have already been visited, simply dequeue 5…



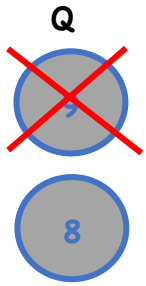
7) Visit 6, and attempt to enqueue all of 6's neighbors, since 5 and 2 have already been visited, simply dequeue 6…



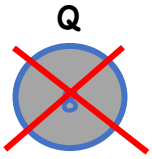
8) Visit 7, and attempt to enqueue all of 7's neighbors, since 4 and 9 have already been visited, we simply enqueue 8, and dequeue 7…



8) Visit 9, and attempt to enqueue all of 9's neighbors, since 4 and 9 and 8 have already been visited, we simply dequeue 9…



8) Visit 8, and attempt to enqueue all of 9's neighbors, since 7 and 4 and 9 have already been visited, we simply dequeue 8…



Shortest Paths:
BFS traversals can further be used to determine the **shortest path** between two vertices
- From our example, it would be the **Shortest Path from 0 to any vertex!**

Analysis of Graph Traversals:
At a high level, DFS and BFS have the same runtime!
1) Each node must be seen and then visited, but the order will differ between these two approaches!

How will the representation of the graph affect the runtimes of these traversal algorithms?

Using an Adjacency Matrix:
Rows/Columns serve as **Vertex Labels:**
1) $M[i][j] = 1$ if $(i, j) \in E$
- If there is an edge between vertices, write 1
2) $M[i][j] = 0$ if $(i, j) \ni E$
- If there is not an edge between vertices, write 0

	0	1	2	3	4	5
0	0	1	0	0	1	0
1	1	0	1	0	1	0
2	0	1	0	1	0	0
3	0	0	1	0	1	1
4	1	1	0	1	0	0
5	0	0	0	1	0	0

Undirected Graph:
1) Mirror across diagonal -> **Symmetric Matrix!**

Adjacency Matrix Analysis:

Runtime:

1) $\theta(1)$: Search for **vertex** (i, j), does it have an edge?
2) $\theta(v)$: To find all neighbors of a vertex!
3) $\theta(v^2)$: To traverse graph!

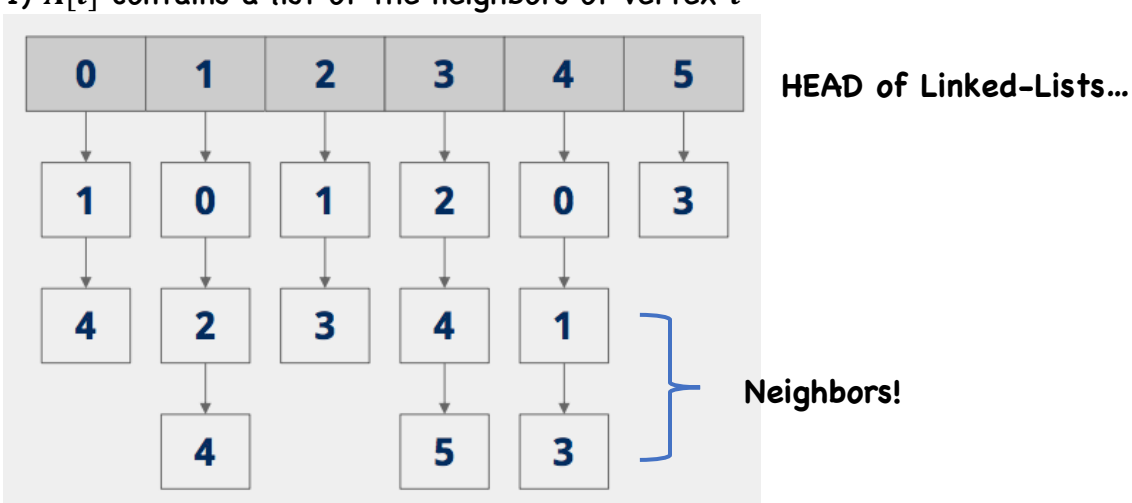
Space: $\theta(v^2)$

- v^2 in memory (v to initialize rows, v to initialize columns)

Adjacency Lists:

An array of **Neighbor Lists!**

1) $A[i]$ contains a list of the neighbors of vertex i



Note: These are not paths!!

Adjacency List Analysis:

Runtime: Depends on the Vertex Degree!

1) $\theta(deg(V))$: Is there an edge?

- Where $deg(V)$ represents the number of neighbors!

2) $\theta(deg(V))$: To find all neighbors to a vertex!

- Where $deg(V)$ represents the number of neighbors!

3) $\theta(V + E)$: To traverse graph!

- Don't drop lower order terms, b/c only with sparse data structure!

Space: $\theta(V + E)$

Comparison:

Adjacency Lists vs. Adjacency Matrices:

1) **Adjacency Lists:**

- Sparse Graph (willing to spend more runtime, save more memory!)

2) **Adjacency Matrix:**

- Dense Graph! (Everything a sparse graph isn't)

3) What about the **List of Nodes/List of Edges Approach?**

NAHHHHHH

DFS & BFS would be Called from a Wrapper Function:

$\theta(V^2)$ **or** $\theta(V+E)$

MATRIX **LIST**

Assuming **Graph** is **Connected:**

1) We're only considering a **SUBSET!**

2) **dfs()/bfs()** is called once, and generates a **Spanning Tree.**

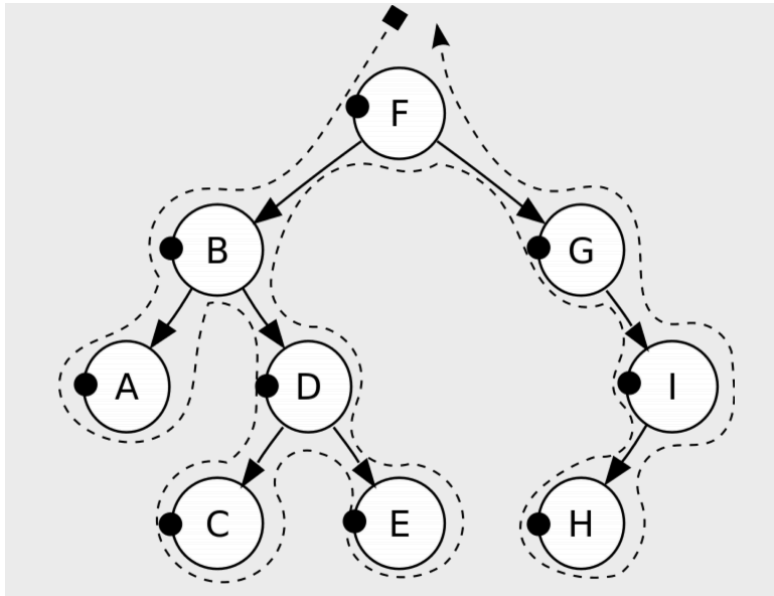
If the **Graph** is **Not Connected:**

1) Loop in wrapper function will continuously be invoked while there are still unseen vertices!

2) Each call will yield a spanning tree for a connected component of the graph.

DFS Pre-Order Traversal:

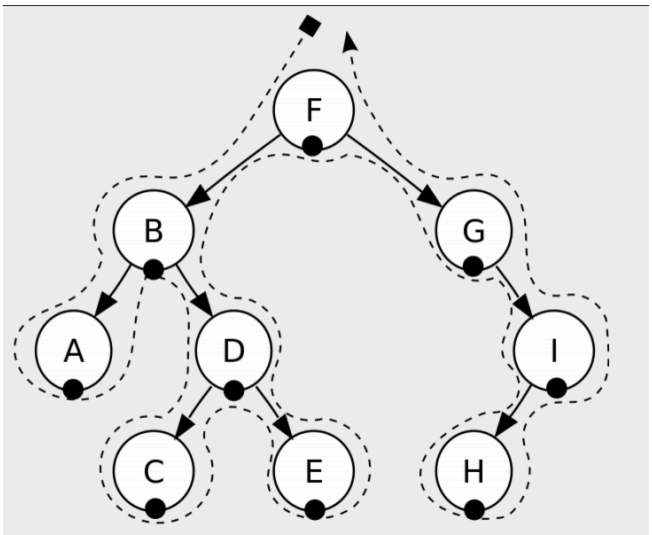
1) Visit vertex (node) before its neighbors (children)



DFS In-Order Traversal:

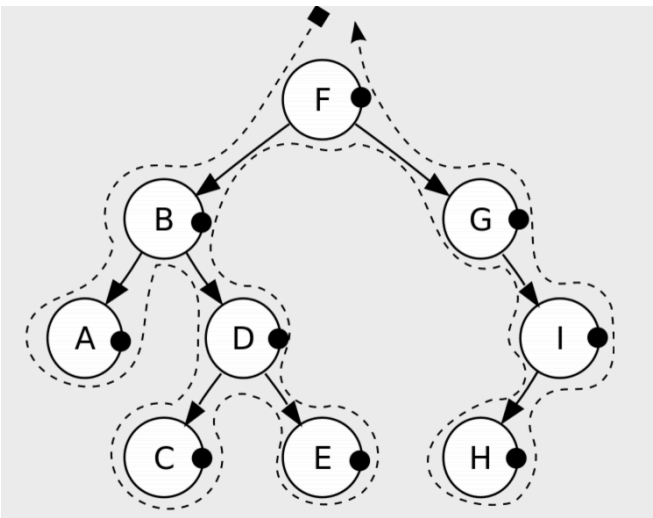
1) Look at the vertex in between neighbors!

- But this is hard to do for graphs!



DFS Post-Order Traversal:

1) Deal w/the Neighbors **(Node)** after **ALL RECURSIVE CALLS**.



Biconnectivity:

1) A **biconnected graph** has **at least 2 distinct paths** (no common edges or vertices) between **all vertex pairs**.

2) Any graph that is not biconnected has one or more **articulation points**
- Vertex that, if disconnected, graphic will be separated

3) Any graph that has **no articulation points** is **biconnected**.
- Thus, we can determine if a graph is **biconnected** if we look for, but do not find any **articulation points**.

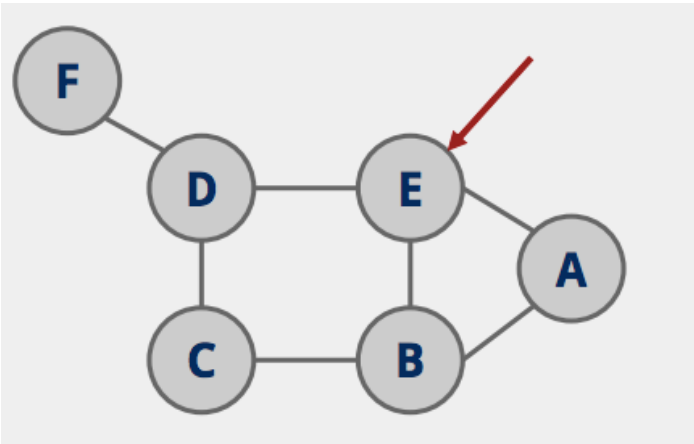
Finding Articulation Points:

Variation on DFS

1) Consider building up the **Spanning Tree**
    a) Have it be **directed**
    b) Create **"back edges"** or **"ghost edges"** when considering a node that has already been visited in constructing the **Spanning Tree**.
    (Tells us more about connectivity)
        bi) **Let** $num(v) =$ **Pre-Order Traversal Order!**
        bii) $low(v) =$ Lowest-Numbered Vertex Reachable from v using zero or more **Spanning Tree edges** and then at most one **Back Edge** (populated in **post-order**)
            biij) **Minimum of:**
                biijk) $num(v)$
                biijkk) **Lowest** $num(w)$ of all **Back Edges** (v, w)
                biijkkk) **Lowest** $low(w)$ of all **Spanning Tree Edges** (v, w)

Example: Finding Articulation Points

Consider the following graph, suppose we start at **Vertex E**, and want to traverse alphabetically…



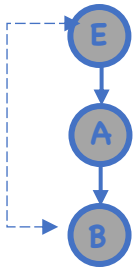
Let's construct the minimum spanning tree!

- Let's keep track of $num(v)$

We start at E, and traverse to its neighbor **A**, since it appears alphabetically before **B**…

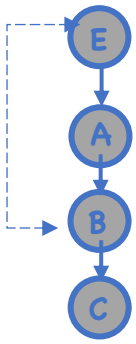
$num(V)$
0
1

Now we start at A, and traverse to its neighbor **B**, since it appears alphabetically before **E**, and observe that **B could have gone to E, so we make a backedge…**



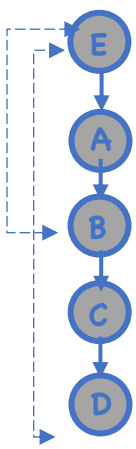
$num(V)$	$low(V)$
0	??
1	??
2	0

Now we start at B, and traverse to its neighbor **C**, since it appears alphabetically before **E**, and observe that **B could have gone to E, so we make a backedge…**



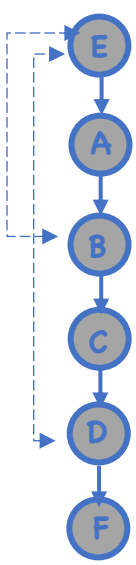
$num(V)$	$low(V)$
0	0
1	0
2	0
3	??

Now we start at C, and traverse to its neighbor **D** since it appears alphabetically before **E**, and observe that **D could have gone to E, so we make a backedge…**



$num(V)$	$low(V)$
0	0
1	0
2	0
3	0
4	0

Now we start at D, and traverse to its neighbor **F** since we've already put a backedge for **E**, and we've already visited **C**



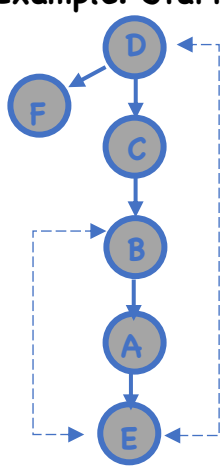
$num(V)$	$low(V)$
0	0
1	0
2	0
3	0
4	0
5	5

We're gonna return from **F to D**
(Since there are no more BEs
Or Unvisited Neighbors!!)

"Post Order" <-> Low values!

Find nodes whose Children can't get higher in the tree than its parent!
1) **D's potential > Parent's Curr State**

So, where are the articulation points?
1) If any (non-root) vertex **v** has some child **w** such that $low(w) \geq num(v)$, **v** is an **articulation point**!

2) **What about if we start at an articulation point?**
- If the root of the Spanning Tree has more than one child, it is an **articulation point!**

Example: Start at Articulation Point